

UNIVERSITÀ TELEMATICA INTERNAZIONALE
UNINETTUNO

FACOLTÀ DI INGEGNERIA

Corso di Laurea Magistrale in

Ingegneria Informatica

ELABORATO FINALE

in

Introduzione ai Big Data

**Reverse engineering dell'algoritmo di ranking di
Amazon**

RELATORE

Prof. Emanuel Weitscheck

CANDIDATO

Sergio Meligrana

ANNO ACCADEMICO 2024/25

Ringrazio la mia famiglia che mi ha supportato anche in questa avventura

Indice

1.	Introduzione	1
1.1	Evoluzione di Amazon	1
1.1.1	Crescita del Fatturato di Amazon (1997–2024)	2
1.1.2	Quantità di Prodotti Venduti su Amazon	3
1.2	Amazon Store.....	3
1.2.1	Architettura a microservizi.....	3
1.2.2	Amazon Web Services (AWS).....	4
1.2.3	Sistema di raccomandazione	4
1.2.4	Motore di ricerca interno.....	5
1.2.5	Automazione con l’AI.....	5
2.	Raccolta dati.....	7
2.1	Scraping.....	9
2.2	Lavori precedenti.....	9
2.2.1	Conversione CSV to MongoDB	10
3.	Elenco dei prodotti	13
3.1	Prodotti periodi 1 e 2.....	13
3.2	Prodotti periodi 3 e 4.....	13
3.3	Caratteristiche raccolte	13
4.	Analisi dei dati	16
4.1	Applicazione per analisi dei dati	16
4.1.1	Architettura.....	16
4.1.2	Implementazione	23
4.1.3	Dettagli dell’interfaccia.....	32
4.1.4	Differenze architetturali con altri lavori.....	36
4.2	Introduzione al Machine Learning	36
4.2.1	Deep learning	37
4.3	Definizione modelli Learn To Rank (LTR)	39
4.3.1	Tipologie di algoritmi.....	39
4.3.2	Selezione degli algoritmi.....	39
4.4	Scelta delle caratteristiche	48
4.5	Analisi dei dati	49
4.5.1	Periodo 1 dal 07/02/2022 al 09/03/2022	49
4.5.2	Periodo 2 dal 21/10/2022 al 11/11/2022	51
4.5.3	Periodo 3 dal 09/09/2024 al 31/12/2024	53
4.5.4	Periodo 4 dal 21/04/2025 al 31/05/2025	56
4.5.5	Analisi complessiva su tutti i periodi	59

4.5.6	Confronto dei risultati tra periodi e variazione nel tempo.....	62
	Conclusioni e sviluppi futuri.....	64
5.	Bibliografia	66

Sommario

L'elaborato mira a comprendere il funzionamento del sistema di ranking dei venditori su Amazon, con un focus particolare sul meccanismo di assegnazione della Buy Box. Attraverso tecniche di scraping, analisi dati e machine learning (Learn To Rank), si tenta di ricostruire, per quanto possibile, l'algoritmo che determina l'ordine dei venditori nella pagina del prodotto.

Nel primo capitolo viene descritta l'evoluzione di Amazon che nasce nel 1994 come libreria online, crescendo fino a diventare uno dei principali player globali dell'e-commerce.

Ha diversificato il proprio business introducendo il Marketplace, Amazon Prime, AWS, Kindle e soluzioni logistiche avanzate che usa per sé stesso e vendo ad altri venditori.

Il fatturato è passato da 148 milioni di dollari nel 1997 a circa 638 miliardi di dollari nel 2024. Amazon ospita oltre 353 milioni di prodotti, considerando anche i venditori terzi.

Nel secondo capitolo vengono descritti i periodi di raccolta dei dati che tramite diverse sessioni fatte da diversi studenti vanno dal 2022 e al 2025. Sono stati analizzati diversi prodotti appartenenti a varie categorie merceologiche (alimentari, tecnologia, illuminazione, ecc.). Lo scraping ha estratto informazioni dalla pagina di dettaglio dei prodotti e dall'elenco degli altri venditori. I dati dei primi due periodi erano stati salvati su Google Sheets e poi durante questa sperimentazione sono stati importati in un database MongoDB tramite un'applicazione scritta in Java Spring Boot.

Nel terzo capitolo vengono descritti i periodi di raccolta dei dati e vengono descritti le caratteristiche raccolte dalla procedura di scraping.

Il quarto capitolo è suddiviso in cinque sottocapitoli principali. Nel primo viene descritta l'architettura del sistema realizzato per analizzare i dati che è composto da più moduli containerizzati e orchestrati con Docker Compose:

- MongoDB + Mongo Express per la gestione dati
- Scraper in Python per l'estrazione

- Backend in Java Spring Boot per l'integrazione
- Frontend in Angular + Bootstrap Italia per l'interfaccia
- Motore ML in Python + FastAPI per l'analisi con algoritmi LTR

Nel secondo viene decritta l'implementazione di dettaglio dei moduli con dettagli tecnologici e argomentazione delle scelte effettuate. Nel terzo viene descritto il funzionamento dell'interfaccia web realizzata. Nel quarto sotto capitolo vengono descritti i modelli di Machine Learning con un focus sul Learn To Rank. I modelli LTR (Learn To Rank), divisi in tre categorie:

- Pointwise (es. Linear Regression, LGBMRegressor)
- Pairwise (es. RankNet su PyTorch)
- Listwise (es. LGBMRanker, CatBoost)

E vengono descritte le caratteristiche dei cinque modelli scelti:

- LGBMRanker
- CatBoost
- PyTorch
- LGBMRegressor
- LinearRegression

Vengono anche indicati quali caratteristiche sono state scelte per formare il dataset di analisi.

Nel quinto sotto capitolo sono riportati i risultati per ognuno dei quattro periodi e dell'analisi complessiva di tutti i dati e vengono descritte le variazioni che sono state trovate tra i vari periodi presi in analisi.

1. Introduzione

Negli ultimi decenni, l'e-commerce ha trasformato radicalmente il modo in cui le persone e le aziende acquistano e vendono prodotti e servizi.

Con il termine e-commerce (o commercio elettronico) si intende l'insieme delle transazioni commerciali che avvengono attraverso Internet, senza la necessità di un contatto fisico diretto tra venditore e acquirente.

L'e-commerce ha mosso i suoi primi passi negli anni '90, con la nascita delle prime piattaforme di vendita online come Amazon ed eBay. Inizialmente percepito come un canale alternativo, ha progressivamente acquisito importanza, fino a diventare una componente strategica fondamentale per ogni tipo di attività commerciale.

L'evoluzione delle tecnologie digitali, l'espansione della connettività a livello globale e l'aumento della fiducia dei consumatori nei confronti degli acquisti online hanno contribuito a un'espansione senza precedenti del settore.

Con il tempo, l'e-commerce ha introdotto nuove modalità di interazione tra venditori e clienti, come i marketplace online, le app mobile di shopping, l'e-commerce B2B (business-to-business), e le soluzioni omnicanale, dove fisico e digitale si integrano per offrire esperienze di acquisto più fluide.

L'avvento di tecnologie avanzate, tra cui l'intelligenza artificiale, la personalizzazione dei contenuti, i sistemi di pagamento digitali, e la logistica intelligente, ha ulteriormente rivoluzionato il settore, rendendo l'esperienza d'acquisto sempre più veloce, personalizzata ed efficiente.

Oggi, l'e-commerce rappresenta una delle principali forze trainanti dell'economia globale, influenzando comportamenti d'acquisto, modelli di business.

1.1 Evoluzione di Amazon

Amazon nasce nel 1994 a Seattle, negli Stati Uniti, fondata da Jeff Bezos.

Originariamente concepita come una libreria online, Amazon ha iniziato la sua attività vendendo libri via Internet, sfruttando la crescente diffusione del web e il suo potenziale commerciale ancora poco esplorato.

L'idea di Bezos era semplice ma rivoluzionaria: offrire un catalogo praticamente illimitato di libri, senza le limitazioni fisiche delle librerie tradizionali.

Il sito, lanciato ufficialmente nel 1995, ottenne subito un enorme successo grazie alla facilità di accesso, ai prezzi competitivi e a un servizio clienti particolarmente attento.

Il motto iniziale di Amazon, "Get Big Fast" (Crescere in fretta), riassumeva perfettamente l'obiettivo dell'azienda: espandersi rapidamente e conquistare il mercato globale.

Negli anni successivi, Amazon ha progressivamente ampliato la sua offerta oltre i libri, includendo CD musicali, DVD, elettronica, abbigliamento, prodotti per la casa e molto altro, diventando un vero e proprio "negozio di tutto" ("Everything Store").

Parallelamente, ha innovato anche nei modelli di business, introducendo:

- Marketplace: consentendo a venditori terzi di proporre i propri prodotti sulla piattaforma.
- Amazon Prime (2005): un abbonamento che offriva spedizioni rapide e gratuite, successivamente ampliato con contenuti streaming.
- Amazon Web Services (AWS) (2006): la divisione cloud computing, che si è rivelata una delle principali fonti di profitto per l'azienda.
- Kindle (2007): il lettore di ebook che ha rivoluzionato il mondo della lettura digitale.

Amazon ha puntato sempre sull'innovazione tecnologica ha introdotto l'uso dell'intelligenza artificiale per le raccomandazioni di prodotto in base ai precedenti acquisti e sulla logistica con magazzini automatizzati.

Ha sperimentato anche i negozi senza casse come Amazon Go.

Oggi, Amazon è uno dei giganti mondiali del commercio elettronico, leader non solo nelle vendite online, ma anche nel settore cloud, nell'intrattenimento digitale, e nella tecnologia.

La sua crescita rappresenta uno dei casi più significativi di evoluzione aziendale legata all'era di Internet, trasformandola da semplice libreria online a ecosistema globale di servizi.

1.1.1 Crescita del Fatturato di Amazon (1997–2025)

Il fatturato di Amazon ha mostrato una crescita costante nel corso degli anni, nel 1997, il fatturato era di circa 148 milioni di dollari mentre nel 2024, ha raggiunto i 637,96 miliardi

di dollari, con un aumento del 10,99% rispetto al 2023. Questa crescita evidenzia l'espansione significativa di Amazon nel settore dell'e-commerce e dei servizi cloud.

Nel Q1 2025, le vendite sono aumentate del 9 % a 155,7 mld \$ con AWS in crescita del 17 % a 29,3 mld \$, Per il Q2 2025, Amazon prevede vendite tra 159–164 mld \$, pari a un incremento annuo tra il 7 % e l'11 %.

1.1.2 Quantità di Prodotti Venduti su Amazon

Amazon offre un vasto assortimento di prodotti, Amazon vende direttamente oltre 12 milioni di prodotti, inclusi libri, media e servizi. Considerando anche i venditori terzi, il numero totale di prodotti disponibili su Amazon supera i 353 milioni. Questi dati sottolineano la vasta gamma di prodotti disponibili sulla piattaforma, contribuendo alla sua posizione dominante nel mercato dell'e-commerce.

1.2 Amazon Store

L'infrastruttura e-commerce di Amazon si basa su una piattaforma tecnologicamente avanzata, progettata per garantire scalabilità, affidabilità e un'elevata esperienza utente. Il portale adotta un modello di marketplace, dove sono presenti prodotti venduti direttamente da Amazon e offerte di terze parti (venditori indipendenti), che sfruttano il portale come canale distributivo. L'integrazione di sistemi di raccomandazione basati su algoritmi di machine learning, insieme a strumenti di personalizzazione e analisi predittiva, consente ad Amazon di offrire un'esperienza di acquisto altamente personalizzata e centrata sull'utente.

Dal punto di vista logistico, Amazon si distingue per un'efficiente rete di magazzini automatizzati, centri di smistamento e hub di consegna, che supportano servizi come Amazon Prime, garantendo spedizioni rapide in tempi ridotti. Il sistema di gestione degli ordini e il tracciamento in tempo reale rappresentano ulteriori punti di forza della piattaforma.

1.2.1 Architettura a microservizi

Amazon ha progressivamente ridisegnato la propria architettura passando da un sistema monolitica a uno a microservizi, utilizzando i microservizi ogni funzionalità del sistema (es. gestione utenti, carrello, ricerca prodotti, pagamento, logistica) è implementata come un servizio autonomo e indipendente. Questo approccio consente:

- Manutenzione modulare e indipendenza dei team di sviluppo.
- Scalabilità ed elasticità orizzontale dei singoli servizi.
- Deploy continuo (CI/CD) e aggiornamenti senza downtime.

1.2.2 Amazon Web Services (AWS)

L'intero ecosistema è ospitato e gestito su Amazon Web Services (AWS), la piattaforma cloud sviluppata internamente. AWS fornisce:

- Compute (EC2, Lambda) per l'esecuzione dei servizi.
- Storage (S3, EBS) per la gestione dei contenuti multimediali e dei dati statici.
- Database per l'archiviazione dei dati strutturati.
- Service mesh e orchestrazione per la gestione e il bilanciamento dei microservizi.

Amazon Web Services (AWS) gestisce un'infrastruttura globale distribuita progettata per garantire alta disponibilità, scalabilità orizzontale e tolleranza ai guasti.

Si basa sui concetti di:

- regione AWS, corrisponde ad un'area geografica
- availability zones, formano le regioni AWS
- data center, indipendenti tra loro formano una availability zones

Tutti e tre questi elementi collaborano per ottenere elasticità e affidabilità. Per questioni di compliance (es. GDPR) è possibile decidere di limitare la scalabilità e la replica dei dati a determinate regioni.

1.2.3 Sistema di raccomandazione

Amazon utilizza algoritmi di machine learning per analizzare il comportamento degli utenti e generare suggerimenti personalizzati. Il sistema combina più tecniche di machine learning, AI e statistica, integrate su larga scala. Le componenti principali includono:

- Collaborative Filtering, sia in modalità User-based confrontando utenti simili per raccomandare prodotti, che Item-based, che analizza le relazioni tra prodotti e le vendite degli stessi, se molti utenti comprano il prodotto X e poi Y, allora Y sarà raccomandato a chi ha comprato X.
- Content-based Filtering, raccomanda in base a caratteristiche del prodotto: categoria, prezzo, brand, descrizione, ecc. analizzando cosa piace al cliente e propone oggetti simili per descrizione o categoria.

- Ranking multi-livello, raccomandazioni basate su sessione, cronologia, contesto, fascia oraria, e un ranking finale che ordina i prodotti in base alla probabilità che vengano comprati, utilizzando modelli basati principalmente su Gradient Boosted Trees, reti neurali profonde.

1.2.4 Motore di ricerca interno

Amazon ha sviluppato un motore di ricerca interno fondamentale nell'esperienza utente e si basa su tecniche di:

- Learning to Rank
- Autocomplete e correzione automatica

Consente agli utenti di trovare prodotti attraverso:

- Query testuali
- Filtri di ricerca
- Ricerca vocale, integrata con Alexa
- Query implicite, guidate dalla navigazione dell'utente

Alle query viene applicata una comprensione semantica tramite modelli di deep learning.

L'indicizzazione dei dati avviene utilizzando dati descrittivi, attributi strutturati e dati comportamentali, per ottimizzare le risposte vengono costruiti degli inverted index.

I risultati vengono presentati al cliente in modo personalizzato basandosi su cronologia acquisti, prodotti visualizzati, categorie preferite, stagionalità o festività locali. Due utenti che effettuano la stessa query potrebbero ottenere risultati differenti.

1.2.5 Automazione con l'AI

Amazon sta adottando strumenti generativi e agenti AI per compiti ripetitivi. Anche i gli addetti che si occupano di AI stanno aumentando. Ha sviluppato una famiglia di modelli LLM chiamati Titan specializzati in vari ambiti.

- Titan Text, modello generativo generalista che supporta chatbot e generazione creativa.
- Titan Embeddings, utilizzato per rappresentazioni vettoriali di testo in ambiti come i motori di ricerca semantica, i RAG (Retrieval Augmented Generation) e la classificazione semantica.
- Titan Image Generator, per generazione di immagini da prompt testuali.

Titan è stato adottato nell'ambito del Helpdesk automatizzati, Analisi e generazione di documentazione, traduzione e sintesi di testi.

Anche il miglioramento di Alexa con l'implementazione di una nuova versione, Alexa+, usa una personalizzata di Titan LLM per risposte naturali, interazione multimodale, automazioni complesse.

2. Raccolta dati

In questa sperimentazione si è deciso di analizzare i prodotti venduti sul sito di Amazon e di raccogliere delle informazioni in modo automatico sul dettaglio di quanti e in che ordine vengono presentati i venditori dello stesso prodotto.

The screenshot shows the Amazon.it product page for a Miracase car phone mount. The product is titled "Miracase Porta Cellulare Auto, [Livello Militare Aspirazione Affidabile] Supporto Telefono Auto Universale, Per il Cruscotto & Bocchette di Aerazione E Parabrezza Adatto a Tutti Smartphone". The price is 11.99€, with a 31% discount from the original price of 17.99€. The product has a 4.4-star rating from 9,368 votes. The page includes a detailed description, a list of compatible devices, and a section for other sellers. A red box highlights the "Altri venditori su Amazon" section, which lists other sellers and their prices.

Altri venditori su Amazon

Venditore	Prezzo	Spedizione
Nuovo (6) da 11.99€	11.99€	Spedizione GRATUITA sul tuo primo ordine

Figura 1 - Amazon: dettaglio prodotto

Nella pagina di dettaglio è sempre presente il venditore scelto da Amazon che definiremo il vincitore della buy box, e se il prodotto viene venduto anche da altri venditori, la possibilità di accedere ad un'altra sezione "altri venditori", evidenziata in rosso nella figura 1.

Miracase Porta Cellulare Auto, [Livello Militare Aspiraz...
 ★★★★★ 9.388 voti
Nuovo
-31% 11⁹⁹€
 Prezzo consigliato: 47,38€ ⓘ
 Risparmia 5% ogni 4 o più [Acquista articoli idonei](#)
[Consegna GRATUITA mercoledì, 30 aprile](#) sul tuo primo ordine.
[Maggiori informazioni](#)
 ▼ Mostra altro

5 opzioni
 ordinati per totale (prezzo + spedizione): dal più basso al più alto Filtra ▼

Nuovo -20% 13⁹⁹€ Prezzo consigliato: 47,38€ ⓘ Spedito da Venditore	Risparmia 5% ogni 4 o più Acquista articoli idonei Consegna GRATUITA mercoledì, 30 aprile sul tuo primo ordine. Maggiori informazioni ... Altro Amazon BODE ★★★★★ (149 valutazioni) 99% positive negli ultimi 12 mesi	Aggiungi al carrello
Nuovo -14% 14⁹⁹€ Prezzo consigliato: 47,38€ ⓘ Spedito da Venditore	Risparmia 5% ogni 4 o più Acquista articoli idonei Consegna GRATUITA venerdì, 2 maggio sul tuo primo ordine. Ordina entro 9 ore 30 min. Maggiori informazioni Amazon Miracase-EU ★★★★★ (832 valutazioni) 99% positive negli ultimi 12 mesi	Aggiungi al carrello

Figura 2 - Amazon: elenco altri venditori

La figura 2 riporta l'elenco degli altri venditori, per ogni venditore sono riportati una serie di informazioni relative al prezzo al nome del venditore e alla modalità di spedizione del prodotto. I venditori vengono presentati con un determinato ordine

In questa sperimentazione verranno raccolti ed analizzati le informazioni del singolo venditore e verrà posta particolare attenzione alla posizione in cui il venditore si trova, considerando questa posizione come un rank che Amazon ha assegnato al venditore per il determinato prodotto.

2.1 Scraping

Lo scraping è il processo automatico di estrazione di dati da una o più pagine web.

Consiste nell'inviare richieste a un sito simulando il comportamento dell'utente sul browser, ricevere il contenuto delle pagine in HTML e poi estrarre le informazioni desiderate in modo strutturato, per poter effettuare analisi successive. Lo scraping simula il comportamento di un utente che visita una pagina, ma lo fa automaticamente aprendo un browser e ripetendo azioni precedentemente salvate. In questa sperimentazione è stata utilizzata la procedura di scraping del lavoro di tesi magistrale

Svelando i segreti della Buy Box in Amazon: un approccio containerizzato. Giulio Baglione

ed è stata eseguita nel periodo dal 21/04/2025 al 31/05/2025.

Per effettuare lo scraping è stata utilizzata la libreria Selenium WebDriver che permette di controllare le operazioni all'interno di un browser in modo programmatico, simulando il comportamento di un utente reale.

È parte della suite Selenium, una delle librerie più usate al mondo per il test automatico delle applicazioni web.

2.2 Lavori precedenti

Una parte dei dati utilizzati durante la fase di analisi di questa sperimentazione derivano da lavori precedenti.

I dati relativi ai periodi

- Il primo lotto di dati va dal 07/02/2022 al 09/03/2022
- Il secondo lotto va dal 21/10/2022 al 11/11/2022

Fanno riferimento all'articolo

An empirical analysis of the most relevant features in the Amazon platform for the Buy Box assignment. Stefano Azzolina, Sergio Meligrana, Manuel Razza, Roger Voyat and Emanuel Weitschek

che affronta il problema di definire quali caratteristiche sono più importanti nella scelta della buy box da parte di Amazon. Nell'articolo ci si focalizza sul vincitore della buy box e

vengono utilizzati diverse tipologie di modelli di machine learning. L'architettura utilizzata prevede l'utilizzo di docker per la suddivisione delle componenti software in unità di esecuzione singole e di Kafka per la gestione di una coda di messaggi con cui vengono sincronizzate le attività e di Fogli Google per il salvataggio dei dati.

I dati del primo lotto sono stati utilizzati anche per il lavoro di testi Triennale

Reverse engineering dell'algoritmo di ranking di Amazon: analisi dell'assegnazione della buy box. Sergio Meligrana

Durante questa sperimentazione erano stati utilizzati i modelli di machine learning CART, RANDOM FOREST e RIPPER ottenendo quelle che le caratteristiche che maggiormente influenzano Amazon nella scelta del venditore da inserire nella buy box sono il numero di valutazione e Spedito da Amazon (FBA).

Per utilizzare i dati in questa sperimentazione le informazioni presenti nei Fogli Google sono state trasformate e salvate all'interno di MongoDB tramite la procedura descritta nel paragrafo 2.2.1

I dati relativi al periodo 09/09/2024 al 31/12/2024 sono stati raccolti durante il lavoro di tesi Magistrale

Svelando i segreti della Buy Box in Amazon: un approccio containerizzato. Giulio Baglione

anche in questo lavoro ci si focalizza sul vincitore della buy box, è stata però creata una dashboard web con python per l'analisi dei dati. I dati raccolti tramite scraping sono stati salvati su mongoDb. L'architettura prevede l'utilizzo di container docker per ogni componente applicativo.

La sperimentazione attuale si concentra sull'ordine in cui i venditori vengono presentati, considerando l'elenco come una classifica, dove ad ogni posizione viene dato un peso. Questo differente approccio permette di analizzare le caratteristiche per capire come influenzano il posizionamento del singolo venditore e non solo quali concorrono al vincitore della buy box.

2.2.1 Conversione CSV to MongoDB

Questa sperimentazione utilizza come database dei dati raccolti MongoDB, visto che una parte dei dati raccolti da lavori precedenti è stata salvata all'interno di Fogli Google è stato necessario portare le informazioni in un unico database.

Per portare i dati raccolti e salvati in Fogli Google è stato scritto un programma specifico.

La prima operazione che è stata effettuata è stata di trasformare i Fogli Google in file csv, formato testuale che semplifica la lettura dei dati.

La tecnologia utilizzata è Java con il supporto del framework Spring Boot.

Spring Boot mette a disposizione configurazioni predefinite per collegarsi a MongoDB tramite l'aggiunta al pom.xml della dipendenza

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

e la definizione nel file application.properties dell'URI di connessione

```
spring.data.mongodb.uri=mongodb://root:amazon@localhost:27017/amazon-
buybox?authSource=admin
```

Per poter scrivere nella collection "Scraping" è stata definita l'interfaccia

```
public interface ScrapingRepository extends MongoRepository<Scraping,
String> { }
```

e la classe Scraping per il mapping della struttura dati

```
@Setter
@Getter
@ToString
@Document("scraping")
public class Scraping {

    @Id
    public String id;

    public String name;
    public String url;
    @Field("datetime_extraction")
    private String datetimeExtraction;
    private Data data;
}
```

Tramite l'annotation @Document viene specificato il nome della collection da utilizzare.

L'utilizzo degli strumenti messi a disposizione da Spring Boot permette di delegare al framework le operazioni di richiamo delle funzionalità di lettura e scrittura di MongoDB.

Per la lettura e il parsing dei file csv si è utilizzata la libreria OpenCsv, per renderla disponibile all'interno del progetto è stata aggiunta la dipendenza

```
<dependency>
  <groupId>com.opencsv</groupId>
  <artifactId>opencsv</artifactId>
  <version>5.10</version>
</dependency>
```

La differenza principale a livello di struttura dati tra i files csv e MongoDB è data dal fatto che nel file csv che per ogni venditore viene scritta una riga che riporta tutte le informazioni mentre su MongoDB viene creato un documento contenente tutti i venditori di quella specifica estrazione. Per aggregare i dati del csv nella struttura del documento MongoDB è stata utilizzato il campo timestamp.

Per generalizzare la quantità di file da importare è stata creata la classe CollectionToImport

```
public class CollectionToImport {
    private String path;
    private String name;
    private String url;
}
```

che permette di impostare i parametri necessari all'import, e successivamente tramite un'iterazione su queste tipologie di oggetti viene richiamata per ogni file la procedura di conversione e importazione su MongoDB.

3. Elenco dei prodotti

I prodotti presi in esame sono diversi in base al periodo in cui sono stati raccolti i dati e alle scelte fatte dagli studenti precedenti. Le categorie merceologiche delle due liste sono però omogenee e comprendono prodotti simile e nel caso delle capsule caffè e delle lampadine la scelta del tipo di prodotto coincide.

3.1 Prodotti periodi 1 e 2

Nel primo e secondo periodo di scraping i prodotti scelti sono:

- Alimentari - Capsule caffè
- Illuminazione - Lampadine Philips
- Sport e tempo libero - Smartwatch - Xiaomi Mi Smart Band 6
- Ufficio - Agenda moleskine

3.2 Prodotti periodi 3 e 4

Nel terzo e quarto periodo di scraping i prodotti scelti sono:

- Tecnologia - Porta cellulare auto
- Tecnologia - Cuffie Gaming
- Tecnologia - Telecamera
- Alimentari - Capsule caffè
- Alimentari – Integratore alimentare
- Tecnologia – Scheda micro SD
- Prodotti per la casa - Insetticida
- Prodotti per la casa – Forno pizza
- Tecnologia – Dispositivo antiabbandono
- Illuminazione – Lampadine Osram
- Prodotti per la casa - Guanti
- Gioco - Cubo di Rubik
- Prodotti per la casa - Insetticida 2

3.3 Caratteristiche raccolte

Le caratteristiche raccolte nel primo e secondo periodo sono

Caratteristica	Descrizione
timestamp	momento di acquisizione della pagina
buy_box	vale 1 se il venditore considerato ha vinto la BuyBox, 0 altrimenti
visibility_order	ordine di comparizione del venditore (parte da 0)
condizione	nuovo o usato
venduto da	venditore
spedito da	chi spedisce
num_valutazioni	numero di valutazioni (feedback) (se presente), altrimenti 0
valutazioni positive (%)	percentuale delle valutazioni positive negli ultimi 12 mesi
prezzo (€)	prezzo unitario
qta_min	quantità minima venduta
prezzo_prod_venduto (€)	prezzo unitario*quantità minima venduta
tipo spedizione	gratuita oppure a pagamento
prezzo spedizione (€)	costo spedizione
prezzo totale (€)	prezzo_prod_venduto (€)*quantità minima venduta
dif_prezzo	prezzo-(prezzo minore) tra tutti i venditori di quel timestamp
dif_prezzo_sped	prezzo spedizione - (prezzo spedizione minore) tra tutti i venditori di quel timestamp
dif_prezzo_tot	prezzo_totale-(prezzo_totale minore) tra tutti i venditori di quel timestamp
g_cons_min	giorni che intercorrono tra il timestamp e il primo giorno di consegna normale
g_cons_max	giorni che intercorrono tra il timestamp e l'ultimo giorno di consegna normale
g_spedizione	giorni che intercorrono tra il primo e l'ultimo giorno di consegna normale
g_cons_vel_min	giorni che intercorrono tra il timestamp e il primo giorno di consegna veloce
g_cons_vel_max	giorni che intercorrono tra il timestamp e l'ultimo giorno di consegna veloce
g_spedizione_vel	giorni che intercorrono tra il primo e l'ultimo giorno di consegna veloce
spedizione_consegna	campo di spedizione consegna (dal quale si estraggono le singole informazioni sulla spedizione e la consegna)
condizione_usato	condizioni di un prodotto usato
stelle	valutazione media in stelle del singolo venditore
delta_consegna	differenza tra la consegna più veloce del venditore corrente e la consegna più veloce tra tutti i venditori
delta_num_val	differenza il numero di valutazioni del venditore corrente e il massimo numero di valutazioni tra tutti i venditori

delta_val_pos (%)	differenza tra la massima percentuale di valutazioni positive e quella del venditore corrente
rapp_piu_basso	rapporto tra il prezzo totale del venditore corrente e il prezzo totale minimo tra tutti i venditori dello stesso timestamp
fba	spedito da Amazon
vend_amazon	venduto da Amazon

Nel terzo e quarto periodo sono state raccolte invece

Caratteristica	Descrizione
Nome Prodotto	Nome del Prodotto
Nome Venditore	Nome del Venditore
Nome Spedizioniere	Nome dello Spedizioniere
Prezzo Prodotto	Prezzo effettivo del prodotto al netto della spedizione
Prezzo Spedizione	Prezzo di spedizione del prodotto
Prezzo Totale	Somma tra il prezzo del prodotto e il prezzo di spedizione
Giorni consegna	Determina la stima di quando il prodotto verrà consegnato al cliente
Numero di valutazioni Prodotto	Numero di recensioni dei clienti per il prodotto
Numero di valutazioni Venditore	Numero di recensioni dei clienti per il venditore
Media valutazioni Venditore	Media delle valutazioni dei clienti da 0 a 5 stelle per il venditore
Percentuale valutazioni positive Venditore	Rappresenta la percentuale di valutazioni positive negli ultimi 12 mesi per il venditore
Nuovo Venditore	Indica se il Venditore è nuovo su Amazon
FBA	Determina se il venditore aderisce o meno alla programma di logistica di Amazon
Amazon come venditore	Verifica se il venditore è Amazon
Differenza prezzo totale da Buy Box winner	Differenza del prezzo totale del prodotto da quello del vincitore della Buy Box
Differenza prezzo prodotto da Buy Box winner	Differenza del prezzo del solo prodotto da quello del vincitore della Buy Box
Differenza prezzo spedizione da Buy Box winner	Differenza del prezzo della sola spedizione da quello del vincitore della Buy Box
Buy Box Winner	Indica se il venditore ha vinto il buy box

4. Analisi dei dati

È stato deciso di analizzare i dati raccolti tramite la procedura di scraping e di interpretare l'elenco dei sellers di un determinato prodotto come una “classifica” di come Amazon considera i venditori. Per provare ad effettuare il reverse engineering di come viene calcolata questa classifica si è deciso di utilizzare modelli di Learn To Rank che considerano i dati in forma raggruppata e ordinata con lo scopo di dare un peso ad ogni caratteristica.

4.1 Applicazione per analisi dei dati

Per agevolare l'analisi dei dati è stato realizzato un software che permette di effettuare scraping e analisi delle informazioni con generazione di tabelle e grafici. Il sistema permette di selezionare range temporali, elenco delle caratteristiche e algoritmi da confrontare.

4.1.1 Architettura

Il sistema di analisi dei dati è stato progettato e realizzato con un'architettura modulare e multi layer, è composto da una interfaccia utente, da un modulo di backend e dal motore di elaborazione.

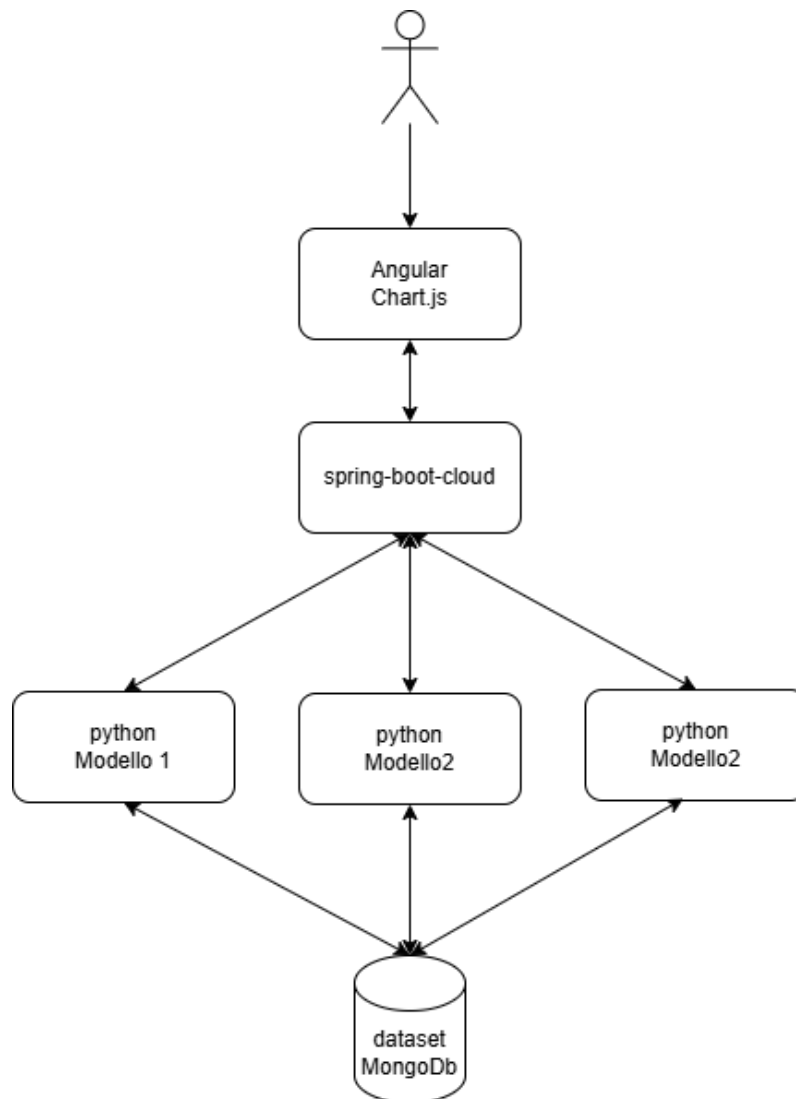


Figura 3 - Architettura applicativa

I moduli applicativi sono stati implementati all'interno di container docker ed orchestrati con docker compose, di seguito lo schema di deploy.

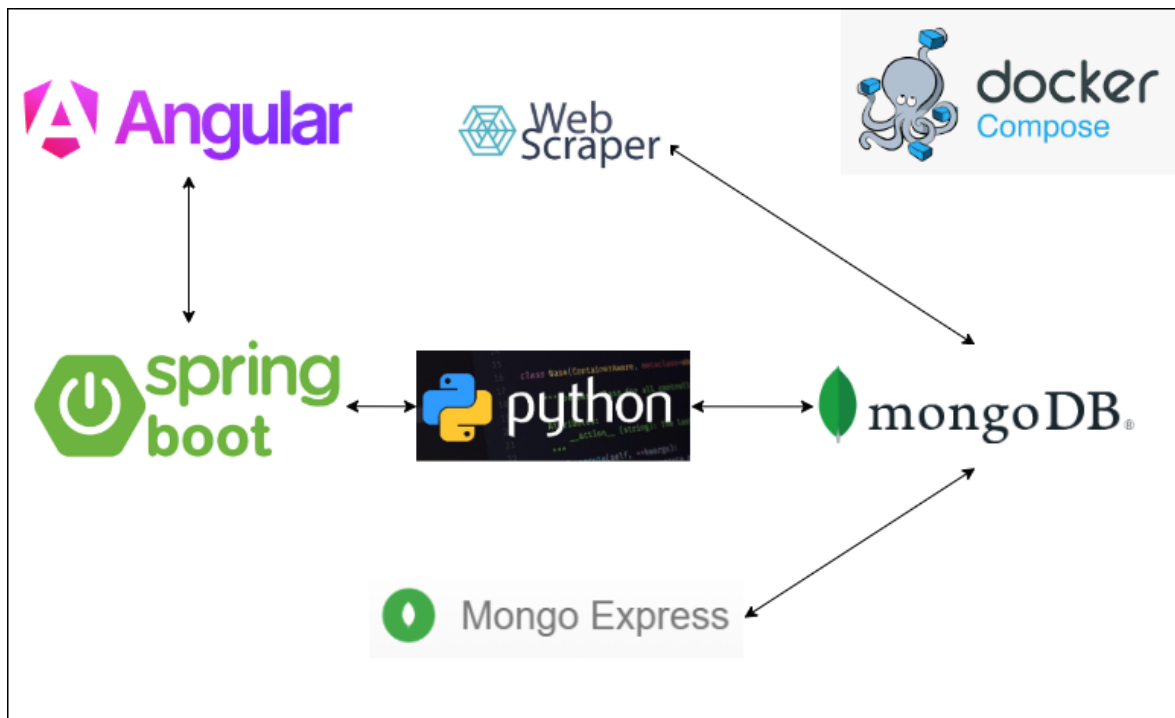


Figura 4 - Architettura container

4.1.1.1 Docker

Docker è una piattaforma open-source che consente la creazione di applicazioni all'interno di container, con lo scopo di automatizzare la distribuzione, il deployment e l'esecuzione. Un container è un'unità leggera, portatile e autosufficiente che include tutto il necessario per eseguire un'applicazione: codice, runtime, librerie di sistema, strumenti e configurazioni. A differenza delle tradizionali macchine virtuali, i container condividono lo stesso kernel del sistema operativo host, risultando così molto più leggeri e veloci da avviare.

Docker si è rapidamente affermato come uno standard de facto nel mondo dello sviluppo e del DevOps, grazie alla sua capacità di rendere il ciclo di vita del software più semplice, ripetibile e scalabile. La sua filosofia si basa sul principio del *"build once, run anywhere"*, ovvero costruire l'applicazione una sola volta e poterla eseguire ovunque, indipendentemente dalla configurazione del sistema operativo o dell'ambiente in cui viene distribuita.

Con i container Docker, la definizione di quanto necessario all'applicazione avviene in modo dichiarativo all'interno del Dockerfile, le informazioni all'interno del file vengono

inserire in un formato di semplice lettura anche per gli umani, l'utilizzo di un file testuale lo rende anche adatto all'utilizzo con sistemi di versionamento come Git, agevolando la tracciatura delle modifiche nel tempo. Tramite strumenti come Docker Compose, è possibile orchestrare facilmente più container per simulare ambienti applicativi complessi.

Sono stati definiti container docker ognuno con uno scopo ben specifico.

4.1.1.2 MongoDB

Per MongoDB è stata utilizzata l'immagine disponibile su docker-hub mongo:latest per avere a disposizione un database senza dover fare nessun tipo di installazione. L'immagine prevede per la creazione delle credenziali dell'utente admin l'utilizzo delle variabili di ambiente.

4.1.1.3 Mongo Express

Per Mongo Express è stata utilizzata l'immagine disponibile su docker-hub mongo-express:latest. Mongo Express permette di avere a disposizione un tool web per l'interrogazione del database, utilizzato per verificarne i contenuti e per verificare le query di estrazione.

4.1.1.4 Scraper

È stato riutilizzato lo script di creazione dell'immagine del precedente lavoro del Dott. Giulio Baglione.

4.1.1.5 Python

Per la parte di esecuzione dei modelli tramite python è stato predisposto un Dockerfile utilizzato per la creazione di un'immagine che include l'eseguibile di python e le librerie necessaria all'esecuzione degli script di analisi dei dati. Per l'installazione delle librerie è stata utilizzata l'utility ufficiale pip.

4.1.1.6 Backend

Sempre con l'utilizzo del DockerFile è stata creata un'immagine che include java 21 e il build del modulo spring-boot a supporto del sistema di frontend e con il ruolo di gateway verso il modulo python. In questo caso il build del codice java è stato effettuato con il supporto di maven. Maven è uno strumento di gestione dei progetti che utilizza un file xml

(pom.xml) per definire la struttura e le dipendenze, semplifica la compilazione, distribuzione e gestione dei progetti java.

4.1.1.7 Frontend

L'immagine creata per il frontend include nginx per servire l'applicazione angular, nginx è stato configurato anche come reverse proxy verso il modulo di backend.

Nginx è un web server ad alte prestazioni, open source, è molto utilizzato perché è leggero, veloce e scalabile. È ampiamente utilizzato come server web ma anche come reverse proxy.

4.1.1.8 Docker compose

Nel codice riportato sotto è possibile osservare come nel file docker-compose.yml, sono presenti i riferimenti alle immagini dei container e i parametri di configurazione degli stessi.

services:

 mongodb:

 image: mongo:latest

 restart: always

 ports:

 - "27017:27017"

 environment:

 - MONGO_INITDB_ROOT_USERNAME=root

 - MONGO_INITDB_ROOT_PASSWORD=amazon

 volumes:

 - mongo-data:/data/db

 mongo-express:

 image: mongo-express:latest

 restart: always

 ports:

 - "8081:8081"

 environment:

 - ME_CONFIG_MONGODB_ADMINUSERNAME=root

 - ME_CONFIG_MONGODB_ADMINPASSWORD=amazon

 - ME_CONFIG_MONGODB_SERVER=mongodb

 depends_on:

```

- mongodb

scraper:
  build:
    context: ./scraper
    dockerfile: Dockerfile.scraper
  depends_on:
    - mongodb
  restart: always
  environment:
    PATH_PRODOTTI_CSV: '/data/scraping/input/prodotti.csv'
    PATH_RESULTS: '/data/scraping/results'
    PATH_LOGS_FILE: '/data/scraping/logs/amazon_data_extraction.log'
  volumes:
    - type: bind
      source: ./data/scraping
      target: /data/scraping
    - type: bind
      source: ./data_extraction/resources/prodotti.csv
      target: /data/scraping/input/prodotti.csv

python:
  build:
    context: ../../python
    dockerfile: Dockerfile
  environment:
    - DB_HOST=mongodb://root:amazon@mongodb:27017/amazon-
      buybox?authSource=admin
  depends_on:
    - mongodb
  ports:
    - "8000:8000"
  restart: always

backend:
  build:
    context: ../../backend/dashboard-gateway

```

```

    dockerfile: Dockerfile
environment:
  - SPRING_PROFILES_ACTIVE=prod
depends_on:
  - python
ports:
  - "8080:8080"
restart: always

frontend:
  build:
    context: ../../frontend/dashboard
    dockerfile: Dockerfile
  depends_on:
    - backend
  ports:
    - "80:80"
  restart: always
  volumes:
    -
      ../../frontend/dashboard/default.conf:/etc/nginx/conf.d/default.conf

volumes:
  mongo-data:
    driver: local

```

È interessante osservare come nella configurazione del container mongodb vengano utilizzate le variabili di ambiente MONGO_INITDB_ROOT_USERNAME e MONGO_INITDB_ROOT_PASSWORD per creare l'utente. Lo stesso meccanismo viene utilizzato nel container mongo-express per configurare l'utente che verrà utilizzato per collegarsi al database e il nome del server. Per quanto riguarda i container scraper, python, backend e frontend, non vengono utilizzate immagini presenti su docker-hub ma vengono create al momento tramite le istruzioni definite negli specifici Dockerfile. Un'altra informazione molto importante è quella relativa ai volumi, ricordando che un container quando viene distrutto e ricreato, ad esempio per un aggiornamento, ritorna allo stato iniziale, perdendo eventuali modifiche fatte ai file al suo interno. Tramite l'utilizzo dei volumi è possibile

rendere persistenti alcune porzioni del filesystem del container, ed è proprio la direttiva `volumes` che permette di ottenere questo risultato. In questo caso è stata utilizzata in due modalità differenti, la prima nella configurazione del container `mongodb mongo-data:/data/db` indica che il volume `mongo-data` verrà reso disponibile all'interno del container nella cartella `/data/db`, la seconda modalità utilizzata nel frontend

```
./../frontend/dashboard/default.conf:/etc/nginx/conf.d/default.conf
```

permette di associare il file `default.conf` della cartella `./../frontend/dashboard` all'interno del container nella cartella `/etc/nginx/conf.d/`.

4.1.2 Implementazione

Sono stati realizzati quattro moduli ognuno con un compito specifico, i moduli sono tecnologicamente indipendenti e dialogano tramite tecnologie standard del cloud come `http` e `rest`, questo permette, nel caso fosse necessario, di sostituire un solo elemento con uno equivalente con l'unica restrizione di dover rispettare le interfacce di comunicazione definite nelle API. L'utilizzo di moduli separati e l'utilizzo di tecnologia a container permette anche di scalare solo alcuni dei componenti e non l'intera applicazione.

4.1.2.1 Frontend: Angular

Angular è un framework open source per lo sviluppo di applicazioni web, creato e mantenuto da Google. Si basa su `TypeScript` e consente di creare `Single Page Applications (SPA)` moderne, modulari e altamente interattive. Angular adotta un approccio strutturato e dichiarativo, favorendo l'organizzazione del codice e la manutenibilità anche in progetti di grandi dimensioni. È un framework completo che include funzionalità di routing, `http client`, gestione dello stato e dei form. Questo riduce la necessità di librerie esterne e garantisce consistenza architetturale agevolando tra l'altro l'aggiornamento del framework. L'utilizzo di `TypeScript`, che estende `javascript`, permette di avere un ambiente con forte tipizzazione la possibilità di definire delle interfacce rendendo il codice più robusto. Inoltre a differenza di `javascript` è compilato permettendo di evidenziare gli errori prima che il codice venga eseguito.

Per la definizione della UX è stato utilizzato il framework `Bootstrap Italia` che è costruito su `Bootstrap` ma con delle personalizzazioni che permettono di rispettare le linee guida di design per i siti internet e i servizi digitali della Pubblica Amministrazione. `Bootstrap Italia`

mette a disposizione componenti pronti all'uso per la realizzazione dell'interfaccia agevolando l'attività di sviluppo e rendendo uniforme le interfacce.

I grafici presenti sulle interfacce sono stati realizzati con Chart.js, una libreria JavaScript open-source che permette di creare grafici interattivi, semplici e belli all'interno di pagine web. È progettata per essere facile da usare, leggera e responsive.

Utilizza il tag <canvas> di HTML5 per disegnare i grafici ed è una delle librerie più popolari nel mondo front-end grazie alla sua semplicità e versatilità.

L'implementazione è stata effettuata creando una pagina che si occupa di renderizzare la dashboard e un componente bar-chart che viene utilizzato dalla dashboard per la presentazione di grafico delle caratteristiche e per quello della trasposta, la dashboard utilizza il componente tramite l'istruzione

```
<app-bar-chart [barChartData]="barChartData"></app-bar-chart>
```

barChartData con cui viene fornito il dataset da visualizzare. Questo approccio permette di avere un componente riutilizzabile, infatti per visualizzare i dati trasposti è sufficiente passare il dataset specifico

```
<app-bar-chart [barChartData]="barChartDataTrasposta"></app-bar-chart>
```

senza dover duplicare il codice che disegna il grafico.

4.1.2.2 Backend: Spring Boot

Spring Boot è un framework open source nato per ridurre la complessità della configurazione e accelerare lo sviluppo di applicazioni Java moderne, scalabili e modulari. I motivi per cui si è scelto di utilizzarlo sono

- Avvio veloce della fase di sviluppo tramite gli starter che permettono di avere le dipendenze necessarie per una specifica funzionalità senza configurazioni manuali.
- Server http embedded senza la necessità di installare un application server
- Integra tutti gli strumenti necessari per realizzare servizi REST
- Adatto al deploy all'interno di container

In questa sperimentazione è stato utilizzato per realizzare i servizi REST richiamati dall'applicativo Angular di frontend, questo modulo richiama i servizi Python che implementano la logica di business. Per ottimizzare e parallelizzare le chiamate è stato

utilizzata la caratteristica nativa di chiamate asincrone. Utilizzare chiamate asincrone permette di sfruttare meglio le risorse hardware a disposizione e di avere un'esecuzione parallela dei tasks.

Per realizzare questa funzionalità è stata creata la classe `AlgorithmExecutorService` e al suo interno è stato realizzato il metodo di esecuzione asincrona.

```
@Async
public CompletableFuture<Result> execute(Algorithm algorithm, String
queryString) {
```

La sola presenza della annotation `@Async` indica al framework che l'esecuzione del metodo deve essere eseguita in un secondo Thread e che la risposta sarà restituita appena disponibile.

Il Thread principale attende la risposta di tutti i Thread tramite la chiamata a `join`

```
CompletableFuture.allOf(results.toArray(new
CompletableFuture<?>[0])).join();
```

e successivamente procede con l'esecuzione. I Threads vengono eseguiti all'interno di un pool che permette di impostare vari parametri. La configurazione del pool di executor è effettuata con la definizione di un `ThreadPoolTaskExecutor`

```
@Bean
public Executor taskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(2);
    executor.setMaxPoolSize(2);
    executor.setQueueCapacity(500);
    executor.setThreadNamePrefix("Dashboard-backend-");
    executor.initialize();
    return executor;
}
```

Di seguito vengono descritti i parametri impostati.

- `CorePoolSize`: Numero minimo di thread che rimangono attivi anche se inattivi.
- `MaxPoolSize`: Numero massimo di thread che il pool può creare se la coda si riempie.
- `QueueCapacity`: Numero massimo di task che possono stare in attesa nella coda se tutti i thread sono occupati.
- `ThreadNamePrefix`: Prefisso usato per nominare i thread

Nella configurazione attuale sono in esecuzione sempre due thread.

Per rendere il sistema flessibile sulla tipologia e quantità di algoritmi da eseguire è stata definita la classe `Algorithm` nella quale vengono parametrizzati le informazioni di esecuzione del singolo algoritmo.

```
public class Algorithm {  
    private final String code;  
    private final String backend;  
}
```

L'informazione di dove risponde l'algoritmo è contenuta nella proprietà `backend`, mentre `code` permette di identificare univocamente l'algoritmo. Questo approccio permette di parallelizzare l'esecuzione e se si decidesse di utilizzare nodi differenti per il deploy dei servizi python si avrebbe la scalabilità orizzontale multinodo. Anche se in questa sperimentazione si è deciso di usare python per l'esecuzione di tutti gli algoritmi, il disaccoppiamento tramite chiamate REST permette di utilizzare tecnologie differenti.

Le configurazioni degli algoritmi sono contenuti all'interno del file `application.properties`

```
algorithms[0].code=LIW-LGBMRanker  
algorithms[0].backend=http://python:8000/api/ltrListwiseLGBMRankerMongo  
algorithms[1].code=LIW-CatBoost  
algorithms[1].backend=http://python:8000/api/ltrListwiseCatBoostMongo  
algorithms[2].code=PAW-Pytorch  
algorithms[2].backend=http://python:8000/api/ltrPairwisePytorchMongo  
algorithms[3].code=POW-LGBMRegressor  
algorithms[3].backend=http://python:8000/api/ltrPointwiseLGBMRegressorMongo  
algorithms[4].code=POW-LinearRegression  
algorithms[4].backend=http://python:8000/api/ltrPointwiseSklearnLinearRegressionMongo  
  
dataExploration=http://127.0.0.1:8000/api/dataExploration  
count=http://127.0.0.1:8000/api/count
```

tramite l'utilizzo dei profili di spring-boot è possibile avere file di configurazioni differenziati per ambiente, utili per gestire l'ambiente di sviluppo differenziato da quello di esercizio e il deploy in un'architettura distribuita. In oltre l'utilizzo di una lista gestita tramite file configurazione permette di aggiungere / rimuovere algoritmi senza dover modificare il codice.

Sono stati resi configurabili anche gli endpoint di data exploration e count sempre per poter rendere flessibile l'applicazione.

L'implementazione dei servizi rest è stata effettuata nella classe DataRestController

```
@RestController
@RequestMapping("/api/data")
public class DataRestController {
```

l'annotation @RestController specifica al framework che deve rendere disponibile le funzionalità come servizi rest e @RequestMapping permette di specificare il path del servizio.

```
@GetMapping("count")
public Integer count(
    @RequestParam(defaultValue = "2000-00-00 00:00:00") String
datetimeExtractionGte,
    @RequestParam(defaultValue = "3000-00-00 00:00:00") String
datetimeExtractionLte)
```

Il metodo count effettua il conteggio dei documenti richiamando lo specifico servizio di backend python, anche in questo caso vengono utilizzate le annotation per istruire il framework, @GetMapping indica che si tratta di un metodo che risponde al verbo http GET e al path count, @RequestParam indica invece che i valori presenti sulla queryString devono essere mappati sulle variabili del metodo.

```
@GetMapping("dataExploration")
public DataExplorationResult dataExploration(
    @RequestParam(defaultValue = "2000-00-00 00:00:00") String
datetimeExtractionGte,
    @RequestParam(defaultValue = "3000-00-00 00:00:00") String
datetimeExtractionLte) {
```

Il metodo dataExploration, reso disponibile sempre in GET, richiama il servizio di backend python che esplora i dati e restituisce i risultati nell'oggetto DataExploracionResult

```
public class DataExplorationResult {
    private Integer count;
    private List<DataExplorationProduct> products;
}
```

Count indica il numero di risultati, mentre products è una lista di DataExplorationResult

```
public class DataExplorationProduct {
    private String name;
    private Integer minIndex;
    private Integer maxIndex;
}
```

che contiene il nome del prodotto il numero minimo e massimo di seller presenti nell'estrazione.

```
@GetMapping
public Collector analyze(
    @RequestParam List<String> columns,
    @RequestParam List<String> algorithms,
    @RequestParam List<String> products,
    @RequestParam(defaultValue = "2000-00-00 00:00:00") String
datetimeExtractionGte,
    @RequestParam(defaultValue = "3000-00-00 00:00:00") String
datetimeExtractionLte,
    @RequestParam(defaultValue = "42") Number seed,
    @RequestParam(defaultValue = "15") Number numEpochs,
    @RequestParam(defaultValue = "100000")
Number visibilityOrderThreshold) throws Exception {
```

Analyze è il metodo che interpreta i parametri ricevuti sulla queryString e richiama i servizi di backend implementati con python che effettuano l'analisi dei dati, il richiamo avviene in modalità asincrona e in parallelo come descritto precedentemente.

4.1.2.3 Business logic: Python

Per ogni algoritmo è stata definita un'interfaccia standard per i parametri di input e per la restituzione dei risultati, per rendere i modelli distribuibili e richiamabile tramite l'implementazione di servizi rest è stata utilizzata la libreria FastApi, un moderno framework web, progettato per la creazione di API REST ad alte prestazioni.

```
@router.get("/ltrListwiseLGBMRankerMongo")
def read_root(columns: List[str] = Query(default=columns),
    products: List[str]= Query(default=[]),
    datetime_extraction_gte= "2000-00-00 00:00:00",
    datetime_extraction_lte= "3000-00-00 00:00:00",
    seed:int=42,
    visibility_order_threshold:int=100000):
```

Tramite l'annotation @router.get viene indicato a FastApi il path da esporre e il verbo http.

In input vengono accettati i parametri:

- columns, per la selezione delle caratteristiche da analizzare;
- products, permette di indicar ei prodotti che devono essere analizzati
- datetime_extraction_gte, per indicare l'inizio del periodo temporale da estrarre dalla base dati
- datetime_extraction_lte, per indicare la fine del periodo temporale da estrarre dalla base dati
- seed, per gestire la randomizzazione della suddivisione degli insiemi di train e test

Per la parte di output sono stati definite due classi python,

```
class Result(BaseModel):  
    ndcg: float  
    featureImportance: list[Feature] = []
```

```
class Feature(BaseModel):  
    indice: int  
    feature: str  
    importance: float
```

Result fa da contenitore per i risultati di dettaglio e riporta il valore di NDCG ottenuto dall'analisi, mentre Feature rappresenta il valore di importanza ottenuto in quella esecuzione dalla specifica feature, all'interno di Result la lista delle features è ordinata per importanza.

I dettagli implementativi dei modelli verranno discussi nel capitolo 4.3.

Il servizio utilizzato per l'esplorazione dei dati effettua una query sul database MongoDB creando una pipeline

```
pipeline = [  
    {"$match": {  
        "datetime_extraction": {  
            "$gte": datetime_extraction_gte,  
            "$lte": datetime_extraction_lte  
        },  
        "data.buybox_seller": {"$exists": True },  
        "data.other_sellers": {"$exists": True },  
    }},  
    {"$group": {  
        "_id": "$name", # Raggruppa per campo 'name'  
        "minIndex": {"$min": {"$size": "$data.other_sellers"}}, # Trova il  
        massimo valore
```

```

    "maxIndex": {"$max": {"$size": "$data.other_sellers"}} # Trova il
    massimo valore
  }},
  {"$sort": {"maxIndex": -1}} # ordina per valore decrescente
]

```

suddivisa in tre step:

1. match: effettua la ricerca filtrando sui dati
2. group: applica le regole di raggruppamento per avere il numero minimo e massimo di seller per ogni prodotto
3. sort: ordina i risultati

L'esecuzione dei singoli modelli condivide una parte di estrazione delle informazioni dalla base dati e di predisposizione del DataFrame. Questa parte è stata definita all'interno del file `utils.py` è richiamata dalle specifiche implementazione delle esecuzioni dei modelli.

La pubblicazione dei servizi REST è stata realizzata tramite la libreria FastApi che permette di definire delle route per specifico servizio, in questa sperimentazione è stata effettuata la scelta di definire un route per ogni modello. L'utilizzo della tecnologia REST permette di avere di abilitare la chiamata ad ogni servizio con qualsiasi tecnologia che supporta i protocolli standard del web aumentando il disaccoppiamento con gli altri moduli e ottenendo anche la possibilità di avere le chiamate parallelizzabili e scalabili su più nodi, anche se in questa sperimentazione si è utilizzato l'approccio a nodo singolo.

4.1.2.4 Database: MongoDB

MongoDB è un database NoSQL orientato ai documenti, progettato per gestire grandi quantità di dati non strutturati o semi-strutturati in modo flessibile e scalabile. A differenza dei database relazionali, non utilizza una struttura a tabelle, ma le informazioni vengono salvate in documenti BSON (Binary JSON), i documenti vengono suddivisi in gruppi chiamati collezioni. Un vantaggio di definire più collezioni è quello di avere un modello dei dati più chiaro, oltre ad avere la possibilità di definire politiche di accesso granulari e aver performance maggiori definendo indici specifici.

Sono state definite due collezioni, la prima chiamata `products` contiene le informazioni necessarie per effettuare lo scraping dei prodotti.

La seconda collezione chiamata `scraping` contiene i risultati dell'estrazione dei dati dal sito di Amazon.

Il singolo documento è organizzato in sezioni:

- Nella root abbiamo:
 - o Name, del prodotto
 - o Url della pagina di dettaglio del prodotto
 - o datetime_extraction, che contiene il timestamp di quando sono state estratte le informazioni
- Nella sezione data abbiamo:
 - o buybox_seller, che indica il vincitore della buy box che nell'analisi sarà l'elemento con la rilevanza maggiore.
 - o other_sellers, l'array de non vincitori di buy box che nell'analisi avranno un valore di rilevanza in base al valore di index che ne indica il posizionamento sulla pagina. Valori di index maggiori indicano una rilevanza minore.

Gli elementi contenuti in buybox_seller e in other_seller condividono la stessa struttura

- nome del venditore
- index, posizionamento nella lista dei venditori proposti
- fba, valorizzato a true se sono stati utilizzati i servizi di logistica offerti da Amazon
- vendita con i sottocampi
 - o priceNoShipping
 - o condition
 - o totalPrice
 - o diffPriceNoShipping
 - o diffTotalPrice
- spedizione con i sottocampi
 - o price
 - o days
 - o diffPrice
- valutazioni con i sottocampi
 - o isNew
 - o avg
 - o num
 - o perc_positive

Avere tutte le informazioni in un unico documento permette di fare delle query molto performanti per estrarre i dati applicando criteri di filtro come la data di estrazione.

4.1.3 Dettagli dell'interfaccia

L'interfaccia realizzata è suddivisa in due parti, la prima permette di impostare i parametri che si vogliono utilizzare per l'analisi dei dati.

The screenshot shows a web dashboard titled "Dashboard" with the subtitle "Analisi dati". Below the title, there is a section "Impostare i parametri da utilizzare per l'analisi". This section contains several form fields and a button:

- Periodo:** Two date pickers labeled "Maggiore o uguale a" and "Minore o uguale a", both showing "gg/mm/aaaa" and a calendar icon. A blue button labeled "Esplora dati" is to the right.
- Prodotti:** A text field with the placeholder "Selezionare **Esplora dati** per caricare la lista dei prodotti".
- Caratteristiche:** A list of checkboxes for features: num_valutazioni, perc_positive, dif_prezzo, dif_prezzo_tot, g_spedizione, prezzo_spedizione, fba, priceNoShipping, totalPrice, and stelle.
- Algoritmi:** A list of checkboxes for algorithms: Pointwise: LGBMRegressor, Pointwise: LinearRegression, Pairwise: Pytorch, Listwise: LGBMRanker, and Listwise: CatBoost.
- Parametri:** Three input fields: "Seed" (value 42), "PyTorch num Epochs" (value 15), and "Visibility Order Threshold" (value 100).

At the bottom of the form is a blue button labeled "Analizza".

Figura 5 - Dashboard

Periodo permette di selezionare il range di date dei campioni da analizzare.

Prodotti permette di selezionare i singoli prodotti da sottoporre ad analisi, la sezione si valorizza dopo avere eseguito "Esplora dati" e dipende dai prodotti presenti nel periodo.

Caratteristiche permette di selezionare i campi che si vogliono analizzare, è possibile selezionare una singola caratteristica fino a tutte senza limiti nelle combinazioni.

Algoritmi permette di impostare i modelli da utilizzare è possibile selezionare un singolo algoritmo o una qualsiasi combinazione.

Parametri permette di impostare i parametri di esecuzione:

- Seed, permette di cambiare la randomicità con cui vengono divisi i dati tra train e test
- Num Epochs, permette di selezionare il numero di epoche per l'algoritmo implementato con PyTorch
- Visibility Order Threshold, indica il valore massimo che deve essere considerato nei seller di un prodotto, i seller con un valore maggiore verranno scartati

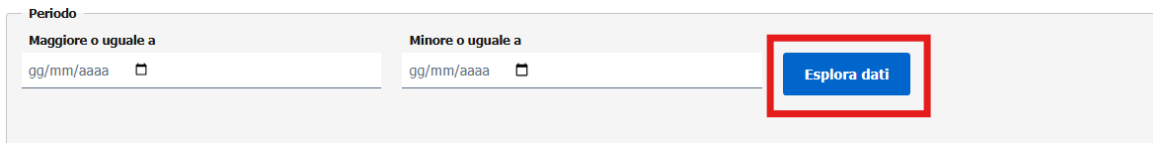


Figura 6 - dashboard: selezione periodo

Il pulsante Esplora dati, evidenziato in rosso nella figura, presente all'interno della sezione Periodo effettua una query con il periodo impostato e restituisce la quantità di dati che verranno estratti durante l'analisi e per ogni prodotto presente nel periodo calcola il numero massimo e minimo di venditori. È utile per valutare preventivamente la dimensione del campione e per parametrizzare l'analisi. Inoltre valorizza la sezione prodotti con l'elenco dei risultati.

Esplorazione dati



Numero di estrazione: 338

Prodotto	Numero minmo di venditori	Numero massimo di venditori
Xiaomi Mi Smart Band 6	71	183
Capsule caffè	32	50
Lampadine Philips	15	36
Agenda moleskine	7	24

Chiudi

Figura 7 - Dabsboard: esplorazione dati

La seconda parte dell'interfaccia permette di visualizzare i risultati dell'analisi ed è divisa in cinque parti

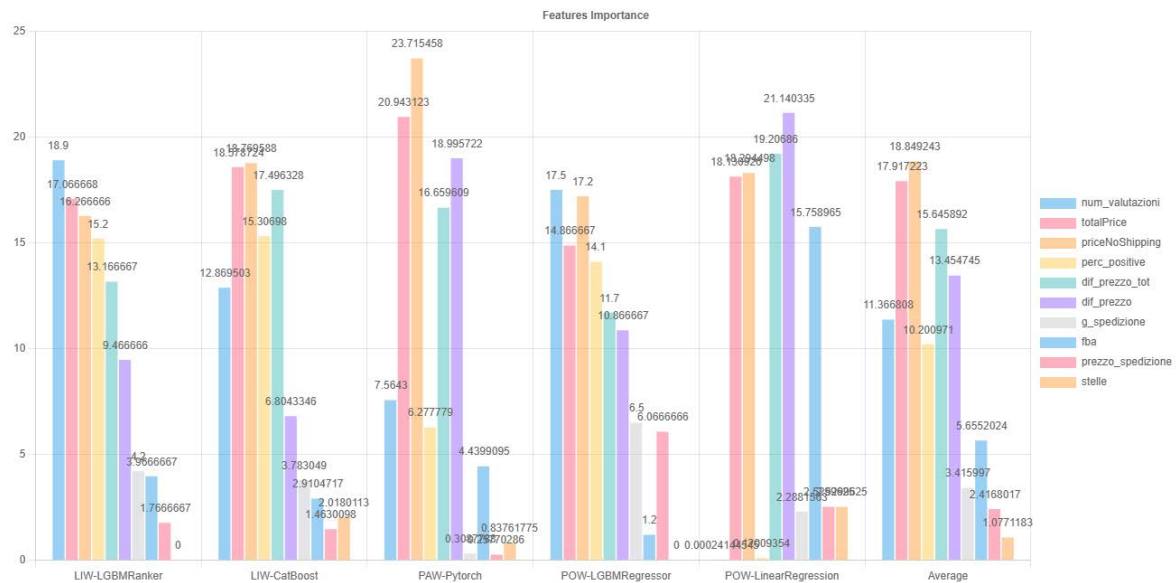


Figura 8 - Dashboard: grafico delle caratteristiche

La prima parte rappresentata dalla figura mostra l'importanza di ogni caratteristica analizzata per algoritmo, la presentazione grafica permette di ottenere un veloce e semplice confronto di come il singolo algoritmo ha dato importanza alle caratteristiche in input. Nella parte destra del grafico è presente la porzione che rappresenta la media dei vari algoritmi.

Caratteristica	LIW-LGBMRanker	LIW-CatBoost	PAW-Pytorch	POW-LGBMRRegressor	POW-LinearRegression	Average
num_valutazioni	18.9	12.869503	7.5643	17.5	0.00024144545	11.366808
totalPrice	17.066668	18.578724	20.943123	14.866667	18.130926	17.917223
priceNoShipping	16.266666	18.769588	23.715458	17.2	18.294498	18.849243
perc_positive	15.2	15.30698	6.277779	14.1	0.12009354	10.200971
dif_prezzo_tot	13.166667	17.496328	16.659609	11.7	19.20686	15.645892
dif_prezzo	9.466666	6.8043346	18.995722	10.866667	21.140335	13.454745
g_spedizione	4.2	3.783049	0.3087788	6.5	2.2881563	3.415997
fba	3.966667	2.9104717	4.4399095	1.2	15.758965	5.6552024
prezzo_spedizione	1.766667	1.4630098	0.25770286	6.0666666	2.5299625	2.4168017
stelle	0	2.0180113	0.83761775	0	2.5299625	1.0771183

Figura 9 - Dashboard: tabella delle caratteristiche

La seconda parte rappresentata dalla figura riporta le stesse informazioni ma in forma tabellare, anche in questo caso è presente la colonna con le medie.

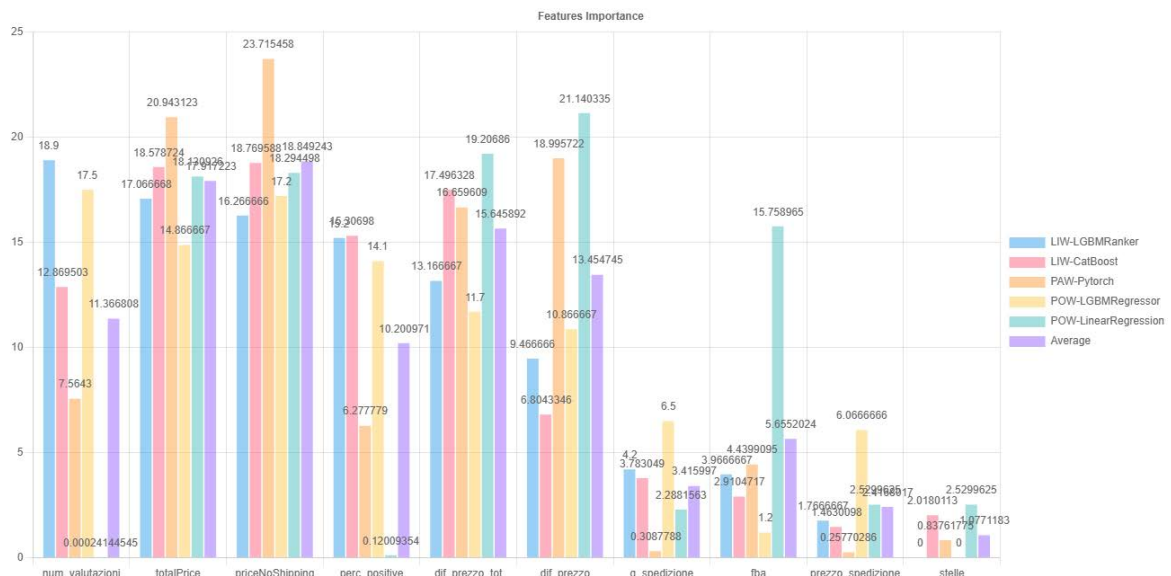


Figura 10 - Dashboard: grafico delle caratteristiche (Trasposto)

La terza parte, visualizzata in figura, è la trasposta dei primi due e permette di visualizzare vicine le caratteristiche e confrontare come il singolo algoritmo le valuta.

Caratteristica	num_valutazioni	totalPrice	priceNoShipping	perc_positive	dif_prezzo_tot	dif_prezzo	g_spedizione	fba	prezzo_spedizione	stelle
LIW-LGBMRanker	18.9	17.066668	16.266666	15.2	13.166667	9.466666	4.2	3.9666667	1.7666667	0
LIW-CatBoost	12.869503	18.578724	18.769588	15.30698	17.496328	6.8043346	3.783049	2.9104717	1.4630098	2.0180113
PAW-Pytorch	7.5643	20.943123	23.715458	6.277779	16.659609	18.995722	0.3087788	4.4399095	0.25770286	0.83761775
POW-LGBMRRegressor	17.5	14.866667	17.2	14.1	11.7	10.866667	6.5	1.2	6.0666666	0
POW-LinearRegression	0.00024144545	18.130926	18.294498	0.12009354	19.20686	21.140335	2.2881563	15.758965	2.5299625	2.5299625
Average	11.366808	17.917223	18.849243	10.200971	15.645892	13.454745	3.415997	5.6552024	2.4168017	1.0771183

Figura 11 - Dashboard: tabella delle caratteristiche (Trasposto)

La quarta parte riporta in formato tabellare le stesse informazioni rappresentate nel grafico a figura.

Ndcg	Valore
LIW-CatBoost	0.99418014
LIW-LGBMRanker	0.9944333
POW-LinearRegression	0.9931883
PAW-Pytorch	0.996438
POW-LGBMRRegressor	0.9990254

Figura 12 - Dashboard: ndcg

La quinta parte riporta in forma tabellare la valutazione NDCG ottenute dagli algoritmi

4.1.4 Differenze architetturali con precedenti lavori

Come indicato nel paragrafo 4.1 il modulo di scraping e le informazioni raccolte sono state riutilizzate da lavoro di tesi

Svelando i segreti della Buy Box in Amazon: un approccio containerizzato. Giulio Baglione

che prevede un'architettura applicativa con l'utilizzo di container docker, i moduli realizzati sono:

- Scraping, per la raccolta delle informazioni
- MongoDB, per il salvataggio in formato json delle informazioni
- Mongo Express, per la consultazione web di MongoDB
- Dashboard, interfaccia ed elaborazione dei dati

La dashboard si occupa sia della parte visuale che della parte di elaborazione degli algoritmi, il tutto scritto con il linguaggio python.

In questa sperimentazione si è voluto evolvere l'architettura aumentandone la scalabilità e la modularità con l'utilizzo di tecnologie eterogenee, motivo per cui si è scelto di suddividere le funzionalità della dashboard in container differenti:

- Frontend, interfaccia utente in Angular
- Backend, con spring-boot come gateway verso i moduli computazionali
- Python, per l'implementazione della parte di elaborazione dei modelli

Si è scelto di utilizzare REST per la comunicazione dei tre nuovi moduli per disaccoppiarli dalla specifica tecnologia implementativa utilizzata dai singoli moduli. Aver scelto di dividere e implementare i moduli python di elaborazione come servizi REST distinti permette di parallelizzare le operazioni e di distribuire orizzontalmente le operazioni di calcolo.

4.2 Introduzione al Machine Learning

Il Machine Learning (ML) è un campo dell'informatica che si occupa di sviluppare algoritmi in grado di apprendere automaticamente dai dati, senza essere esplicitamente programmati.

Questi algoritmi costruiscono modelli matematici a partire da dati di esempio, per effettuare previsioni o prendere decisioni senza intervento umano diretto.

Il Machine Learning può essere suddiviso principalmente in due categorie:

- algoritmi supervisionati, il modello viene addestrato su un insieme di dati etichettato, ovvero ogni esempio nei dati contiene sia l'input (elenco di caratteristiche) sia l'output corretto (la classificazione). L'obiettivo è imparare una funzione che, dato un nuovo input, riesca a prevedere correttamente l'output associato. Esempi di algoritmi supervisionati:
 - o Classificazione: prevedere una categoria (es. riconoscimento di email spam o non spam).
 - o Regressione: prevedere un valore continuo (es. previsione del prezzo di una casa).
- algoritmi non supervisionati, i dati non sono etichettati. Il modello cerca quindi di scoprire strutture, relazioni o pattern nascosti nei dati, senza avere indicazioni esplicite su cosa dovrebbe trovare. Esempi di algoritmi non supervisionati:
 - o Clustering: raggruppare dati simili insieme (es. segmentazione dei clienti in base ai comportamenti di acquisto).
 - o Riduzione della dimensionalità: semplificare i dati mantenendone la struttura principale (es. PCA - Principal Component Analysis).

4.2.1 Deep learning

Il Deep Learning è un sottoinsieme del Machine Learning che utilizza reti neurali artificiali profonde per apprendere automaticamente rappresentazioni complesse dai dati.

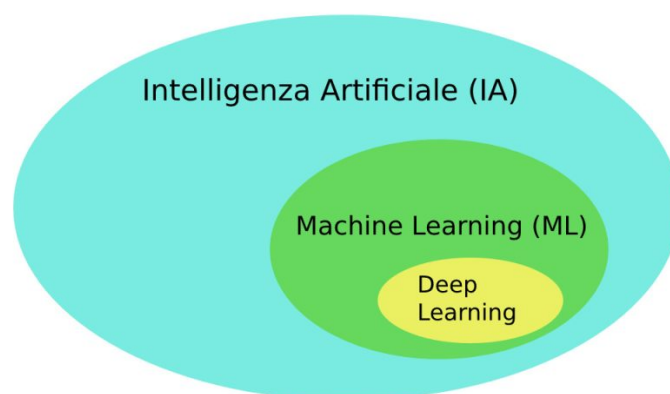


Figura 13 - Rapporto ML- DL

È un metodo per insegnare alla macchina a "capire" dati grezzi senza bisogno di definire manualmente le regole. Una rete neurale è un modello matematico ispirato al funzionamento del cervello umano è composta da nodi (detti neuroni artificiali) organizzati in strati (layer):

- Strato di input: riceve i dati in ingresso.
- Strati nascosti: elaborano i dati attraverso operazioni matematiche.
- Strato di output: restituisce il risultato finale (es. una classificazione o una previsione).

Ogni connessione tra neuroni ha un peso, e ogni neurone applica una funzione di attivazione (es. ReLU, sigmoid) per decidere se e quanto "attivarsi". Le reti neurali sono molto potenti per riconoscere pattern complessi nei dati.

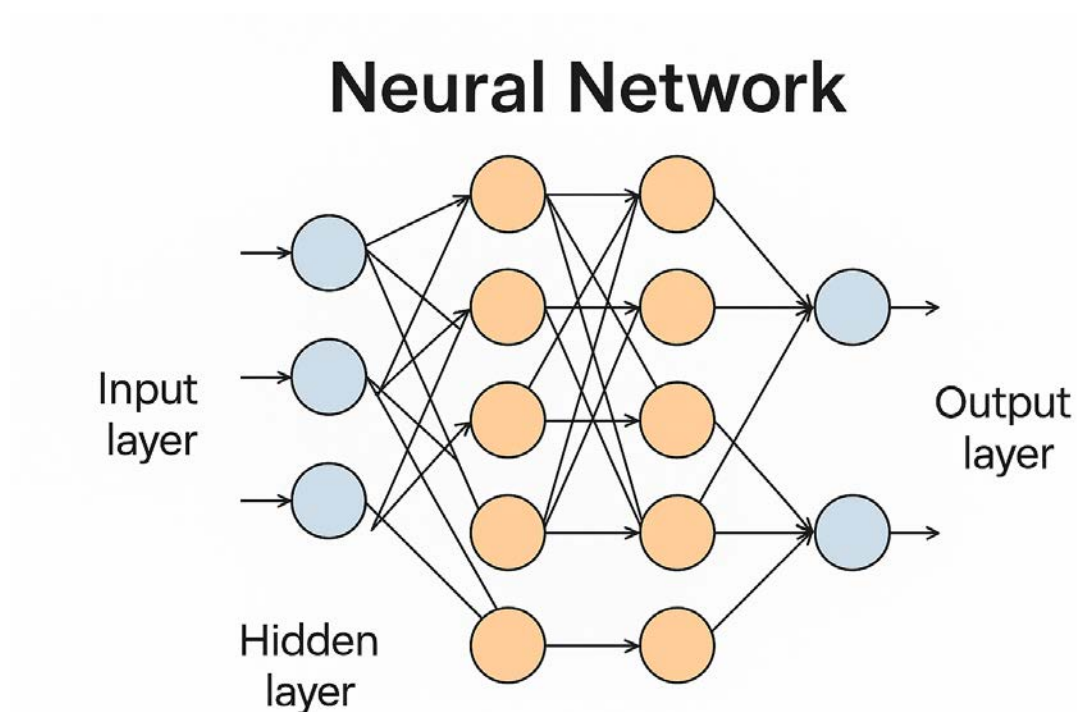


Figura 14 - Rete neurale

Ogni strato è composto da neuroni connessi tramite pesi. Durante l'addestramento:

- La rete fa una previsione
- Calcola l'errore (loss)
- Regola i pesi usando un algoritmo chiamato back propagation con gradient descent

Le difficoltà che si incontrano nell'utilizzo di reti neurali sono dovute principalmente a:

- Richiede grandi quantità di dati e potenza di calcolo.

- È spesso una black box: difficile capire esattamente "cosa ha imparato".
- Può sovradattarsi ai dati se non gestito bene (overfitting).

4.3 Definizione modelli Learn To Rank (LTR)

Il Learn to Rank (LTR) è una tipologia di apprendimento automatico supervisionato che si occupa di ordinare gli elementi in base alla loro rilevanza rispetto a una determinata query di raggruppamento. È utilizzato soprattutto nei motori di ricerca, nei sistemi di raccomandazione, pubblicità online, e-commerce e ogni ambito in cui lo scopo è avere un elenco di risultati ordinato in base alla rilevanza. A differenza dei tradizionali modelli di classificazione o regressione, l'obiettivo di Learn to Rank non è semplicemente etichettare o predire un valore, ma definire un ordinamento ottimale tra un insieme di item. Il modello apprende da dati etichettati con la rilevanza e raggruppati tramite la query. La rilevanza acquisisce un significato nel momento in cui viene considerata insieme ai dati del raggruppamento. È importante considerare i raggruppamenti in ogni fase, quando sono stati divisi i dati tra train e test è stato fatto considerando i raggruppamenti e anche nella fase di cross validation i raggruppamenti sono stati mantenuti.

4.3.1 Tipologie di algoritmi

LTR può essere suddiviso in tre principali categorie di algoritmi:

- Pointwise, è un algoritmo che tratta ogni elemento individualmente e cerca di predire un punteggio che rappresenta la sua rilevanza. È simile alla regressione o classificazione, ma con l'obiettivo finale di ordinamento.
- Pairwise, è un algoritmo che costruisce il modello confrontando coppie di item per decidere quale sia più rilevante rispetto all'altro. La funzione di perdita è definita sul numero di ordinamenti errati.
- Listwise, è un algoritmo che considera l'intera lista di item raggruppati per query e apprende dall'ordinamento. Le funzioni di perdita sono progettate per ottimizzare metriche di ranking come NDCG.

4.3.2 Selezione degli algoritmi

La scelta degli algoritmi da utilizzare per analizzare è stata fatta scegliendo almeno un algoritmo per tipologia, in modo da poter osservare come i vari approcci si comportano. Tutti i modelli sono valutati tramite la metrica Normalized Discounted Cumulative Gain (NDCG)

che valuta la qualità dell'ordinamento tenendo conto della posizione, perché NDCG possa valutare correttamente la bontà del modello è necessario che i dati di testing siano correttamente raggruppati. I raggruppamenti sono stati effettuati tramite

```
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
splits = list(gss.split(x, y, groups))
```

GroupShuffleSplit permette di effettuare lo split dei dati mantenendo la consistenza dei gruppi, il gruppo non viene mai spezzato tra i due insiemi di train e di test. Il gruppo di train viene utilizzato per addestrare il modello mentre quelli di test per validarlo. Questo approccio è necessario, per cercare di evitare l'overfitting e per poter stimare le prestazioni del modello valutandolo su dati nuovi che non ha mai visto.

Tramite GroupShuffleSplit è stata effettuata la suddivisione del dataset con 80% di dati di training e 20% di dati per il test. Con questa suddivisione i dati utilizzati per il test sono dati nuovi per il modello permettendo di valutare il comportamento su dati reali. Un buon comportamento con i dati di test permette di escludere l'overfitting del modello.

Per valutare i modelli si è scelto di utilizzare NDCG che misura quanto bene è ordinata una lista di risultati in base alla rilevanza degli item, dando più peso a quelli che si trovano in cima alla lista.

È una metrica che considera sia la rilevanza dei risultati, sia la loro posizione nel ranking.

Per valutare con NDCG il comportamento dei modelli è stata effettuata una predizione sul campione di test

```
y_pred = model.predict(test_pool)
```

e le predizioni ottenute sono state verificate confrontandole con i valori veri del campione tramite la funzione `ndcg_score` applicandola sui dati raggruppati per query e calcolandone poi la media

```
ndcgs = []
for true, pred in zip(y_test_groups, y_pred_groups):
    if len(true) >= 2:
        ndcg = ndcg_score([true], [pred], k=10)
        ndcgs.append(ndcg)

mean_ndcg = sum(ndcgs) / len(ndcgs)
print(f"Media NDCG@10: {mean_ndcg:.4f}")
```

```
ndcg = mean_ndcg
```

4.3.2.1 LGBMRanker, algoritmo della famiglia Listwise

LGBMRanker è un algoritmo di apprendimento supervisionato progettato per risolvere problemi di Learning to Rank (LTR), è basato su alberi decisionali, è possibile utilizzarlo con tutti e tre le categorie di LTR, in questa sperimentazione è stato utilizzato impostando "lambdarank" che utilizza listwise. Durante il training per ogni coppia di documenti nello stesso gruppo, si calcola la differenza di ranking atteso. Se un documento dovrebbe essere sopra un altro (in base alla label), si calcola un gradiente chiamato "lambda" per spingere l'ordine corretto. LGBMRanker usa questi lambda per ottimizzare i gradienti e addestra alberi per ridurre gli errori tra il ranking predetto e quello desiderato. Lo schema general ed esecuzione prevede:

1. Calcolo del gradiente del loss listwise per ogni documento.
2. Utilizzo dei gradienti calcolati per costruire un nuovo albero di decisione.
3. Aggiunta dell'albero al modello per correggere gli errori del passo precedente.

Con la costruzione di ogni albero si cerca di migliorare la capacità del modello di ordinare correttamente le liste.

Per lo scopo di questa sperimentazione è stato prima di addestrare il modello è stato creato l'oggetto di configurazione

```
model = lgb.LGBMRanker(  
    objective="lambdarank",  
    metric="ndcg",  
    boosting_type="gbdt",  
    n_estimators=100,  
    learning_rate=0.1,  
    label_gain=np.arange(1, 200)  
)
```

Impostando i parametri

Parametro	Descrizione
objective="lambdarank"	Specifica l'obiettivo di ottimizzazione
metric="ndcg"	Indica la metrica usata per valutare il modello: NDCG (Normalized Discounted Cumulative Gain)

boosting_type="gbdt	Usa Gradient Boosted Decision Trees, il metodo standard di boosting.
n_estimators=100	Numero di alberi da addestrare.
learning_rate=0.1	Tasso di apprendimento: quanto ogni nuovo albero corregge gli errori dei precedenti
label_gain=label_gain	Definisce i pesi dei livelli di rilevanza dei dati per calcolare il guadagno cumulativo. È utile per il calcolo di metriche

Dopo si è proceduto ad addestrare il modello

```
callbacks = [lgb.reset_parameter(learning_rate=lambda i: 0.05)]
model.fit(
    x_train, y_train,
    group=group_sizes_train,
    eval_set=[(x_test, y_test)],
    eval_group=[group_sizes_test],
    eval_metric="ndcg",
    callbacks=callbacks
)
```

oltre ad impostare i gruppi di train e di eval è stata definita la funzione di callbacks che imposta ad ogni iterazione il learning_rate a 0.05.

Come ultima fase tramite

```
y_pred = model.predict(x_test)
```

sono state effettuate le predizioni per procedere con la fase di valutazione uguale per tutti i modelli.

4.3.2.2 CatBoost, algoritmo della famiglia Listwise

CatBoost (abbreviazione di Categorical Boosting) è un algoritmo di machine learning supervisionato progettato per fornire prestazioni elevate nelle attività di classificazione, regressione e ranking. Si basa sul concetto di gradient boosting su alberi decisionali. Costruisce il modello in maniera incrementale, combinando tanti decision tree per ottenere un modello forte.

Lo schema generale di esecuzione prevede:

1. Preprocessing automatico dei dati categorici.
2. Costruzione di una sequenza di alberi usando gradient boosting.
3. Aggiornamento iterativo per minimizzare la funzione di perdita.

4. Valutazione con metriche di apprendimento.

L'addestramento del modello è stato effettuato tramite la chiamata a

```
model = CatBoostRanker(  
    iterations=1000,  
    learning_rate=0.1,  
    depth=6,  
    loss_function='QuerySoftMax',  
    random_seed=seed  
)
```

Parametro	Descrizione
iterations=1000	Numero di alberi. Impostando un valore maggiore di numero di alberi si avrà un modello più complesso.
learning_rate=0.1	Tasso di apprendimento: controlla quanto ogni nuovo albero contribuisce alla correzione dell'errore.
depth=6	Profondità massima di ciascun albero decisionale. Va scelto un valore che sia un compromesso tra capacità di apprendimento e rischio di overfitting.
loss_function='QuerySoftMax'	Funzione di perdita per il ranking. QuerySoftMax è progettata per ottimizzare l'ordinamento dei documenti all'interno di gruppi
random_seed=seed	Imposta la casualità per rendere l'esperimento riproducibile

terminata la fase di predisposizione del modello è stato effettuato l'addestramento utilizzando il data set di train

```
model.fit(train_pool)
```

e successivamente è stata effettuata la predizione sui dati di test

```
y_pred = model.predict(test_pool)
```

per poter procedere con la fase di valutazione.

4.3.2.3 Pytorch, algoritmo della famiglia Pairwise

PyTorch è un framework open-source per il machine learning e il deep learning è ampiamente usato per creare, addestrare e ottimizzare reti neurali e altri modelli di apprendimento automatico. Supporta all'accelerazione GPU tramite CUDA.

In questa sperimentazione è stata creata una rete neurale

```

class RankNet(nn.Module):
    def __init__(self, input_dim):
        super(RankNet, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1) # Un solo output (score di ranking)
        )

    def forward(self, x):
        return self.model(x)

```

che si basa sull'architettura di RankNet, la rete non predice direttamente una classe o un valore numerico, ma un punteggio di ranking: i dati vengono confrontati a coppie e l'obiettivo è apprendere un ordinamento tra gli elementi.

In input abbiamo il numero di caratteristiche che si vogliono analizzare e in output un solo valore che corrisponde al valore di rilevanza predetto.

I livelli Linear sono fully connected, ReLu è la funzione di attivazione serve a introdurre non linearità, Dropout spegne casualmente dei neuroni, percentuale specificata nel parametro, per evitare overfitting.

L'addestramento del modello è avvenuto per epoch (iterazioni)

```

for q in unique_queries:
    idx = np.where(query_train == q)[0]
    if len(idx) < 2: # Saltiamo gruppi con un solo elemento
        continue

    x_batch = x_train_tensor[idx]
    y_batch = y_train_tensor[idx]

    optimizer.zero_grad()
    y_pred = model(x_batch).squeeze()

    loss = pairwise_loss(y_pred, y_batch)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5)
    optimizer.step()

```

```
total_loss += loss.item()
```

la quantità di epoch da eseguire sono state impostate con un parametro esterno configurabile.

La funzione di perdita (pairwise_loss) è stata definita nel seguente modo

```
def pairwise_loss(y_pred, y_true):
    y_pred = y_pred.view(-1)
    y_true = y_true.view(-1)

    # Calcolo le differenze di etichette e predizioni
    diff_labels = y_true.unsqueeze(1) - y_true.unsqueeze(0)
    S_ij = torch.sign(diff_labels)

    pred_diff = y_pred.unsqueeze(1) - y_pred.unsqueeze(0)

    mask = S_ij != 0
    S_ij_masked = S_ij[mask]
    pred_diff_masked = pred_diff[mask]

    loss = torch.nn.functional.softplus(-S_ij_masked * pred_diff_masked)
    return torch.mean(loss)
```

opera per coppie calcolando le differenze tra i valori label e quelli predetti, e fa uso di softplus, se il valore è basso predizione corretta altrimenti errata, come ultimo passaggio viene restituito il valore medio delle perdite.

Quando si opera con le reti neurali è complesso riuscire a dare un peso alle singole caratteristiche, per interpretare il comportamento della rete è stata usata la tecnica SHAP (SHapley Additive exPlanations) che si basa sulla teoria dei giochi cooperativi, in particolare sul concetto di valori di Shapley, che misura quanto ogni "giocatore" (cioè, ogni feature) contribuisce al risultato finale (la predizione).

Viene prima inizializzato l'explainer utilizzando un sotto insieme dei dati

```
explainer = shap.KernelExplainer(model_predict,
x_train[np.random.choice(x_train.shape[0], 200, replace=False)])
```

e poi calcoliamo i valori di Shapley

```
shap_values =
explainer.shap_values(x_train[np.random.choice(x_train.shape[0], 50,
replace=False)])
```

per riuscire ad ottenere una valutazione del modello e l'importanza delle caratteristiche.

4.3.2.4 LGBMRegressor, algoritmo della famiglia Pointwise

LGBMRegressor è il modello di regressione di LightGBM (Light Gradient Boosting Machine), una libreria di boosting basata su alberi di decisione, sviluppata da Microsoft. È progettata per essere veloce, efficiente in memoria e altamente scalabile anche su grandi dataset. In questa sperimentazione è stata utilizzata per prevedere un valore numerico continuo trattandosi di una previsione sul singolo sample non è necessario raggruppare i dati in base alla query_id.

Il modello è stato configurato tramite la chiamata a

```
model = lgb.LGBMRegressor(objective="regression", n_estimators=100,  
learning_rate=0.1)
```

Parametro	Descrizione
objective="regression"	Specifica che il modello è per regressione
n_estimators=100	Numero di alberi. Più alto è il valore maggiore sarà la complessità del modello
learning_rate=0.1	Tasso di apprendimento: indica quanto ogni nuovo albero contribuisce alla correzione dell'errore dei precedenti

successivamente viene allenato il modello

```
model.fit(x_train, y_train)
```

per passare poi alla fase di predizione

```
y_pred = model.predict(x_test)
```

necessaria per l'attività di valutazione.

4.3.2.5 LinearRegression, algoritmo della famiglia Pointwise

LinearRegression è uno degli algoritmi fondamentali nel Machine Learning supervisionato, utilizzato per risolvere problemi di regressione. L'obiettivo è trovare una relazione lineare tra una o più variabili indipendenti (input) e una variabile dipendente (output). L'obiettivo è trovare una linea retta (nel caso più semplice) che approssimi al meglio i dati. In questa sperimentazione è stato utilizzato come LGBMRegressor per prevedere un valore numerico

continuo nella categoria dei pointwise anche in questo caso non è necessario raggruppare in base alla query_id. Lo schema generale di esecuzione prevede:

1. Inizializzazione: si parte con valori casuali per i coefficienti.
2. Predizione: si calcola $\hat{y}$ (output predetto) per ogni esempio.
3. Calcolo dell'errore: si confronta $\hat{y}$ con il valore reale y usando ad esempio MSE (Mean Squared Error)
4. Ottimizzazione: si usa un metodo (come gradient descent) per aggiornare i coefficienti e minimizzare l'errore.
5. Iterazione: si ripetono i passi precedenti fino a convergenza.

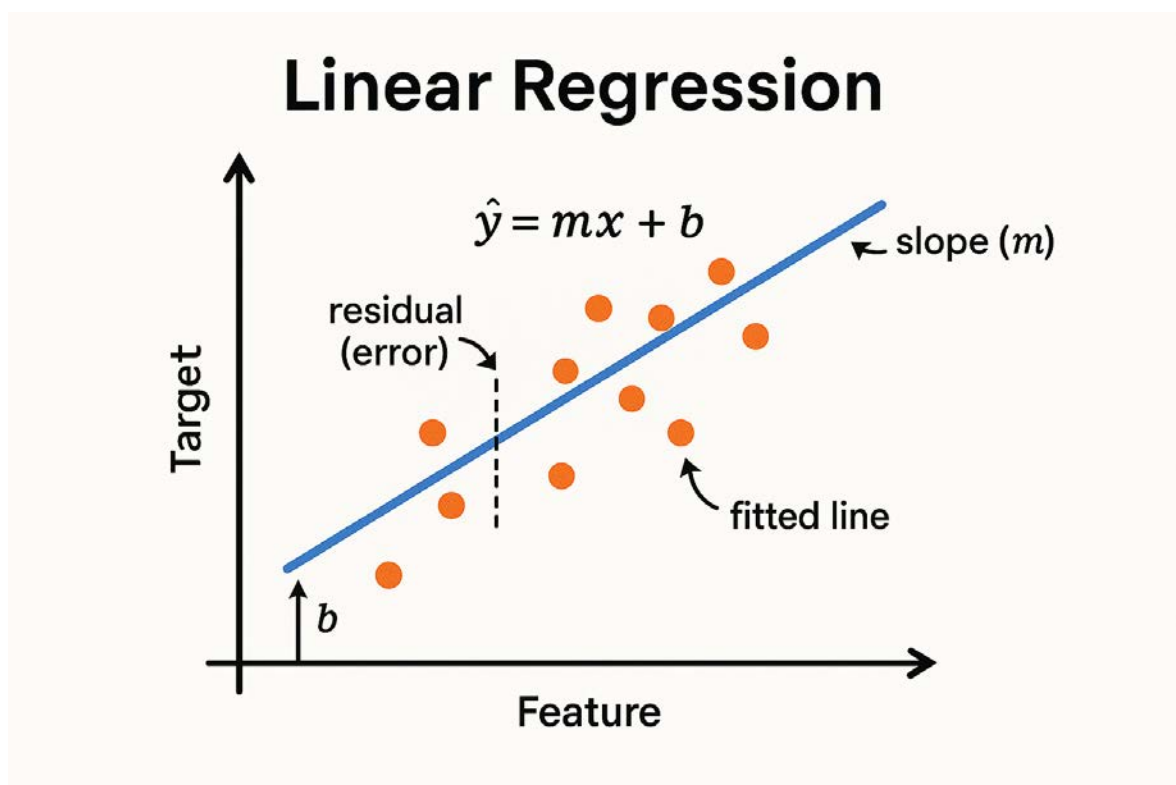


Figura 15 - Linear regression

La creazione del modello è stata effettuata tramite l'operazione

```
model = LinearRegression()
```

l'addestramento tramite

```
model.fit(x_train, y_train)
```

per eseguire poi l'ultima fase di predizione per raccogliere le informazioni necessarie alla valutazione

```
y_pred = model.predict(x_test)
```

4.4 Scelta delle caratteristiche

Della totalità delle caratteristiche, descritte nel paragrafo 1.1.1, sono state selezionate per essere presentate nell'interfaccia di analisi

- Numero di valutazione venditore
- Percentuale di valutazioni positive venditore
- Media delle valutazioni dei clienti da 0 a 5 stelle per il venditore
- Differenza prezzo prodotto
- Differenza prezzo prodotto totale
- Prezzo effettivo del prodotto al netto della spedizione
- Prezzo totale dato dalla somma tra il prezzo del prodotto e il prezzo di spedizione
- Giorni consegna del prodotto
- Prezzo di spedizione
- FBA indica se il venditore aderisce o meno al programma di logistica di Amazon

Mentre come caratteristiche utilizzate a supporto degli algoritmi ma non presenti in interfaccia abbiamo

- `datetime_extraction`, come chiave di raggruppamento
- `visibility_order`, utilizzato per calcolare la rilevanza

Il campo `visibility_order` estratto durante la fase di scraping presenta i venditori in ordine crescente, quindi il vincitore della buybox ha valore zero il secondo venditore ha valore 1 e così a crescere, per l'analisi effettuata era necessario avere invece in valore più alto per il venditore con la maggiore rilevanza per ottenere questo è stata applicata la funzione

```
y = df["visibility_order"]
max_visibility_order = y.max()
y = max_visibility_order - y
```

4.5 Analisi dei dati

L'analisi dei dati è stata effettuata sui singoli periodi:

- Periodo 1 dal 07/02/2022 al 09/03/2022
- Periodo 2 dal 21/10/2022 al 11/11/2022
- Periodo 3 dal 09/09/2024 al 31/12/2024
- Periodo 4 dal 21/04/2025 al 31/05/2025

È stata effettuata anche un'analisi sulla totalità dei dati. Per tutte le analisi è stata utilizzata la dashboard descritta nel paragrafo 6.1.

4.5.1 Periodo 1 dal 07/02/2022 al 09/03/2022

Tramite le funzionalità offerte dalla dashboard è stato analizzato il periodo dal 07/02/2022 al 09/03/2022.

Esplorazione dati dal 07/02/2022 al 09/03/2022

Numero di estrazione: 240

Prodotto	Numero minimo di venditori	Numero massimo di venditori
Xiaomi Mi Smart Band 6	118	179
Capsule caffè	32	49
Lampadine Philips	15	35
Agenda moleskine	7	14

Figura 16 - Periodo 1: esplorazione dei dati

Come mostrato in figura 16, sono presenti 240 registrazioni e quattro prodotti diversi, l'Agenda moleskine è il prodotto che ha il minor numero di venditori, mentre Xiaomi MI Smart Band 6 quello con il maggior numero.

È stata eseguita l'analisi su tutte le caratteristiche e con tutti e cinque gli algoritmi. Parametrizzando il sistema come da figura 17

Seed	PyTorch num Epochs	Visibility Order Threshold
42	15	100

Figura 17 - Periodo 1: parametri

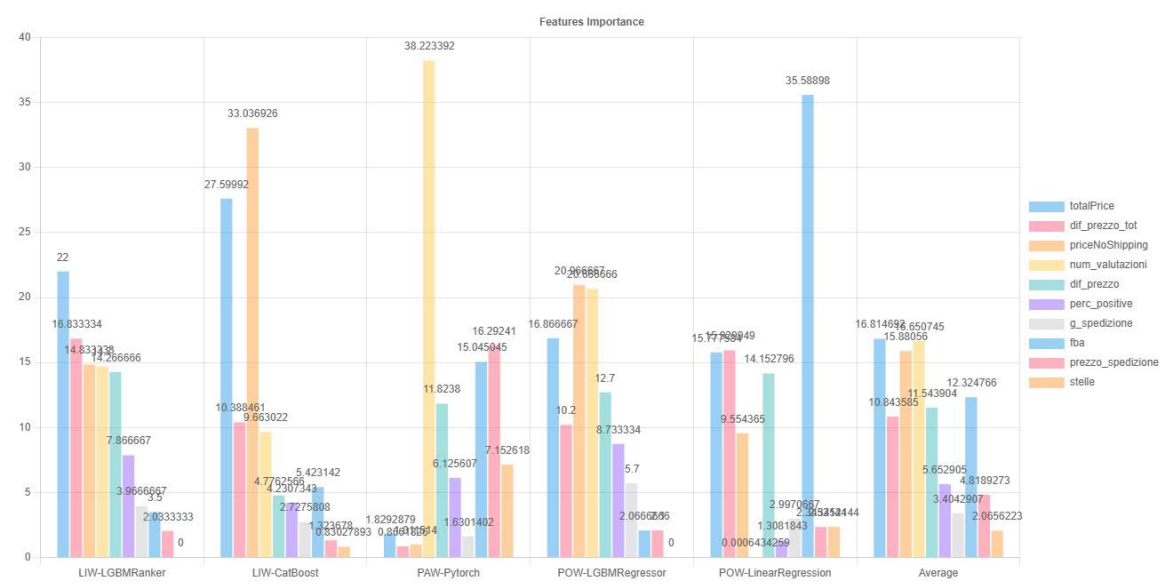


Figura 18 - Periodo 1: grafico delle caratteristiche

Caratteristica	LIW-LGBMRanker	LIW-CatBoost	PAW-Pytorch	POW-LGBMRRegressor	POW-LinearRegression	Average
totalPrice	22	27.59992	1.8292879	16.866667	15.777584	16.814692
dif_prezzo_tot	16.833334	10.388461	0.8661826	10.2	15.929949	10.843585
priceNoShipping	14.833333	33.036926	1.011514	20.966667	9.554365	15.88056
num_valutazioni	14.7	9.663022	38.223392	20.666666	0.0006434259	16.650745
dif_prezzo	14.266666	4.7762566	11.8238	12.7	14.152796	11.543904
perc_positive	7.866667	4.2307343	6.125607	8.733334	1.3081843	5.652905
g_spedizione	3.966667	2.7275808	1.6301402	5.7	2.9970667	3.4042907
fba	3.5	5.423142	15.045045	2.0666666	35.58898	12.324766
prezzo_spedizione	2.0333333	1.323678	16.29241	2.1	2.3452144	4.8189273
stelle	0	0.83027893	7.152618	0	2.3452144	2.0656223

Figura 19 - Periodo 1: tabella delle caratteristiche

La figura 18 mostra sotto forma di grafico le informazioni che sono riportate in forma tabellare nella figura 19.

Ndcg	Valore
LIW-CatBoost	0.9941574
LIW-LGBMRanker	0.9936648
POW-LinearRegression	0.9873135
PAW-Pytorch	0.9935499
POW-LGBMRegressor	0.99883574

Figura 20 - Periodo 1: NDCG

In figura 20 vengono presentati i risultati NDCG. Tutti gli algoritmi hanno ottenuto risultati superiori al 98%.

4.5.2 Periodo 2 dal 21/10/2022 al 11/11/2022

Tramite le funzionalità offerte dalla dashboard è stato analizzato il periodo dal 21/10/2022 al 11/11/2022.

Esplorazione dati dal 21/10/2022 al 11/11/2022

Numero di estrazione: 83

Prodotto	Numero minimo di venditori	Numero massimo di venditori
Xiaomi Mi Smart Band 6	71	100
Capsule caffè	36	42
Lampadine Philips	16	36
Agenda moleskine	16	23

Figura 21 - Periodo 2: esplorazione dati

Come mostrato in figura, sono presenti 83 registrazioni e quattro prodotti diversi, l'Agenda moleskine insieme alle lampadine Philips sono i prodotti che hanno il minor numero di venditori, mentre Xiaomi MI Smart Band 6 quello con il maggior numero.

È stata eseguita l'analisi su tutte le caratteristiche e con tutti e cinque gli algoritmi. Parametrizzando il sistema come da figura 22

Seed	PyTorch num Epochs	Visibility Order Threshold
56	15	100

Figura 22 - Periodo 2: parametri

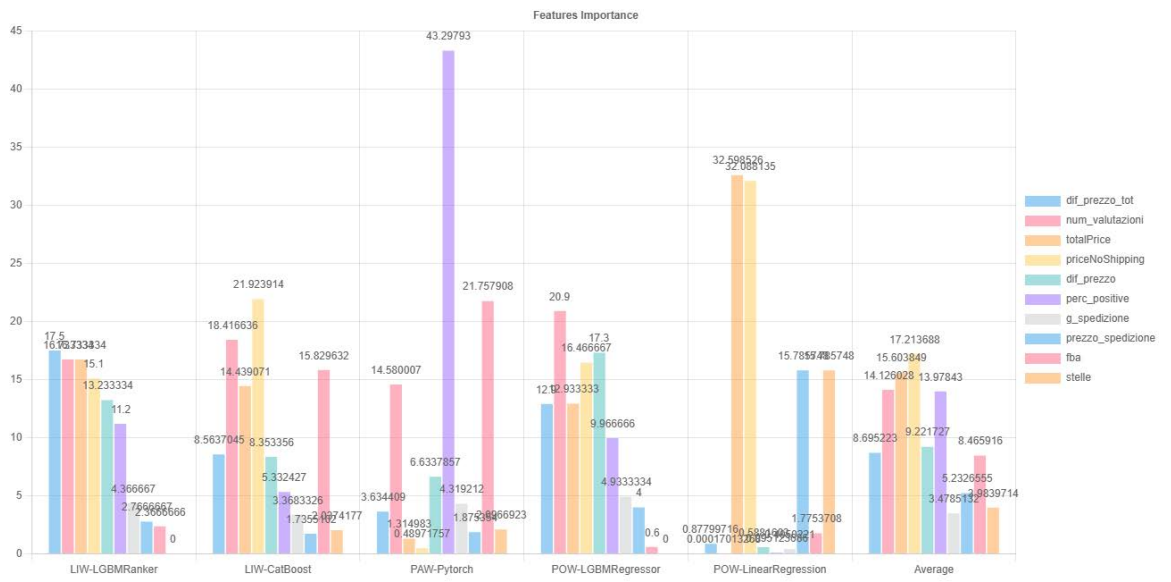


Figura 23 - Periodo 2: grafico delle caratteristiche

Caratteristica	LIW-LGBMRanker	LIW-CatBoost	PAW-Pytorch	POW-LGBMRegressor	POW-LinearRegression	Average
dif_prezzo_tot	17.5	8.5637045	3.634409	12.9	0.87799716	8.695223
num_valutazioni	16.733334	18.416636	14.580007	20.9	0.00017013268	14.126028
totalPrice	16.733334	14.439071	1.314983	12.933333	32.598526	15.603849
priceNoShipping	15.1	21.923914	0.48971757	16.466667	32.088135	17.213688
dif_prezzo	13.233334	8.353356	6.6337857	17.3	0.5881603	9.221727
perc_positive	11.2	5.332427	43.29793	9.966666	0.095123686	13.97843
g_spedizione	4.366667	3.3683326	4.319212	4.9333334	0.4050221	3.4785132
prezzo_spedizione	2.766667	1.7355102	1.875354	4	15.785748	5.2326555
fba	2.3666666	15.829632	21.757908	0.6	1.7753708	8.465916
stelle	0	2.0374177	2.0966923	0	15.785748	3.9839714

Figura 24 - Periodo 2: tabella delle caratteristiche

La figura 23 mostra sotto forma di grafico le informazioni che sono riportate in forma tabellare nella figura 24.

Ndcg	Valore
LIW-CatBoost	0.99444014
LIW-LGBMRanker	0.99268246
POW-LinearRegression	0.98035324
PAW-Pytorch	0.99179316
POW-LGBMRegressor	0.9984284

Figura 25 - Periodo 2: NDCG

In figura vengono presentati i risultati NDCG. Tutti gli algoritmi hanno ottenuto risultati superiori al 98%.

4.5.3 Periodo 3 dal 09/09/2024 al 31/12/2024

Tramite le funzionalità offerte dalla dashboard è stato analizzato il periodo dal 09/09/2024 al 31/12/2024.

Esplorazione dati dal 09/09/2024 al 31/12/2024



Numero di estrazione: 30697

Prodotto	Numero minimo di venditori	Numero massimo di venditori
TELECAMERA	10	11
FORNO PIZZA	6	11
SCHEDA MICRO SD	10	11
INTEGRATORE ALIMENTARE	10	11
LAMPADINE	10	11
GUANTI	10	11
CUBO DI RUBIK	10	11
INSETTICIDA 2	10	11
CUFFIE GAMING	10	11
DISPOSITIVO ANTIABBANDONO	10	11
CAPSULE CAFFE'	3	10
PORTA CELLULARE AUTO	0	6
INSETTICIDA	0	5

Figura 26 - Periodo 3: esplorazione dati

Come mostrato in figura 23, sono presenti 30.697 registrazioni e 13 prodotti diversi, porta cellulare auto insieme a insetticida sono i prodotti che hanno il minor numero di venditori, vengono presi in considerazione i prodotti che hanno tra i 10 e gli 11 venditori. I prodotti porta cellulare auto, insetticida, capsule caffè e forno pizza vengono esclusi dall'analisi, utilizzando la funzionalità specifica come mostrato in figura 24, perché hanno una quantità ridotta di venditori.

Prodotti

☒ INSETTICIDA 2
 ☒ TELECAMERA
 ☒ INTEGRATORE ALIMENTARE
 ☒ SCHEDA MICRO SD
 ☐ FORNO PIZZA
 ☒ LAMPADINE
 ☒ GUANTI
 ☒ CUBO DI RUBIK
 ☒ DISPOSITIVO ANTIABBANDONO
 ☒ CUFFIE GAMING
 ☐ CAPSULE CAFFE'
 ☐ PORTA CELLULARE AUTO
 ☐ INSETTICIDA

Figura 27 - Periodo 3: prodotti

È stata eseguita l'analisi su tutte le caratteristiche e con tutti e cinque gli algoritmi. Parametrizzando il sistema come da figura 28

Seed	PyTorch num Epochs	Visibility Order Threshold
89	15	100

Figura 28 - Periodo 3: parametri

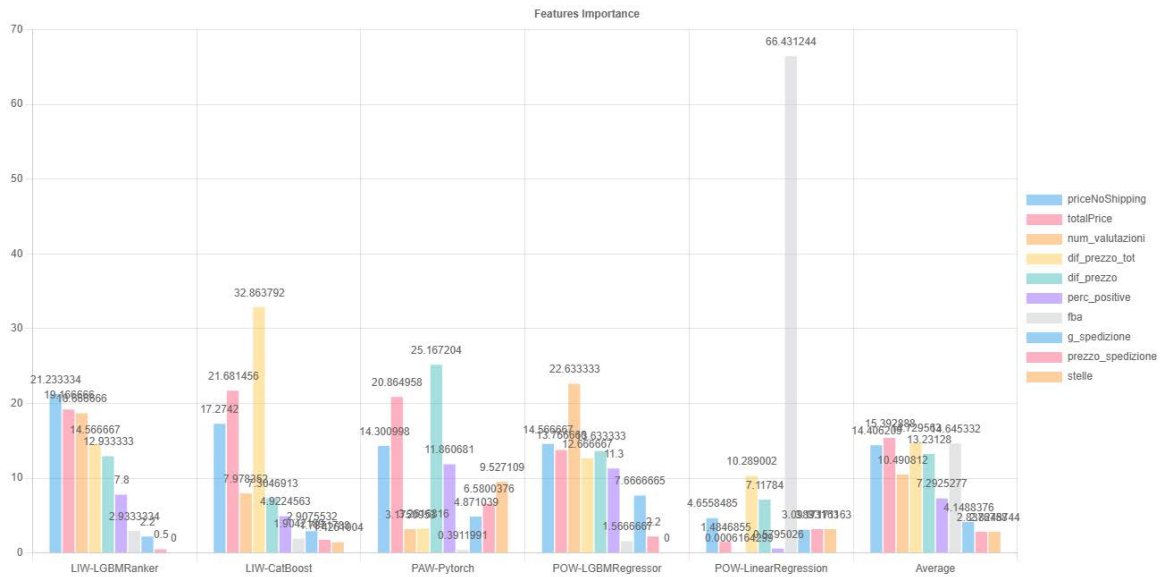


Figura 29 - Periodo 3: grafico delle caratteristiche

Caratteristica	LIW-LGBMRanker	LIW-CatBoost	PAW-Pytorch	POW-LGBMRRegressor	POW-LinearRegression	Average
priceNoShipping	21.233334	17.2742	14.300998	14.566667	4.6558485	14.406209
totalPrice	19.166666	21.681456	20.864958	13.766666	1.4846855	15.392888
num_valutazioni	18.666666	7.978352	3.1750953	22.633333	0.0006164299	10.490812
dif_prezzo_tot	14.566667	32.863792	3.2616816	12.666667	10.289002	14.729563
dif_prezzo	12.933333	7.3046913	25.167204	13.633333	7.11784	13.23128
perc_positive	7.8	4.9224563	11.860681	11.3	0.5795026	7.2925277
fba	2.9333334	1.9042189	0.3911991	1.5666667	66.431244	14.645332
g_spedizione	2.2	2.9075532	4.871039	7.6666665	3.098931	4.1488376
prezzo_spedizione	0.5	1.7371788	6.5800376	2.2	3.171163	2.8376758
stelle	0	1.4261004	9.527109	0	3.171163	2.8248744

Figura 30 - Periodo 3: tabella delle caratteristiche

La figura 29 mostra sotto forma di grafico le informazioni che sono riportate in forma tabellare nella figura 30.

Ndcg	Valore
LIW-CatBoost	0.9971929
LIW-LGBMRanker	0.9967578
POW-LinearRegression	0.9832581
PAW-Pytorch	0.9988933
POW-LGBMRegressor	0.99927807

Figura 31 - Periodo 3: NDCG

In figura vengono presentati i risultati NDCG. Tutti gli algoritmi hanno ottenuto risultati superiori al 98%.

4.5.4 Periodo 4 dal 21/04/2025 al 31/05/2025

Tramite le funzionalità offerte dalla dashboard è stato analizzato il periodo dal 21/04/2025 al 31/05/2025.

Esplorazione dati dal 21/04/2025 al 31/05/2025



Numero di estrazione: 832

Prodotto	Numero minimo di venditori	Numero massimo di venditori
INTEGRATORE ALIMENTARE	10	11
GUANTI	10	11
FORNO PIZZA	8	11
DISPOSITIVO ANTIABBANDONO	10	11
CUFFIE GAMING	10	11
SCHEDA MICRO SD	10	11
CUBO DI RUBIK	10	11
LAMPADINE	10	11
INSETTICIDA 2	10	11
CAPSULE CAFFE'	3	7
PORTA CELLULARE AUTO	5	6
INSETTICIDA	0	4
TELECAMERA	1	3

Figura 32 - Periodo 4: Esplorazione dati

Come mostrato in figura 32, sono presenti 832 registrazioni e 13 prodotti diversi, telecamera insieme a insetticida sono i prodotti che hanno il minor numero di venditori, prendiamo in considerazione i prodotti che hanno tra i 10 e gli 11 venditori. I prodotti porta cellulare auto, insetticida, capsule caffè, telecamera e forno pizza vengono esclusi dall'analisi, utilizzando la funzionalità specifica come mostrato in figura 33, perché hanno una quantità ridotta di venditori.

Prodotti

☒ INSETTICIDA 2
 ☒ INTEGRATORE ALIMENTARE
 ☒ SCHEDA MICRO SD
 ☐ FORNO PIZZA
 ☒ LAMPADINE
 ☒ GUANTI
 ☒ CUBO DI RUBIK
 ☒ DISPOSITIVO ANTIABBANDONO
 ☒ CUFFIE GAMING
 ☐ PORTA CELLULARE AUTO
 ☐ CAPSULE CAFFE'
 ☐ INSETTICIDA
 ☐ TELECAMERA

Figura 33 - Periodo 4: prodotti

È stata eseguita l'analisi su tutte le caratteristiche e con tutti e cinque gli algoritmi. Parametrizzando il sistema come da figura 34

Seed	PyTorch num Epochs	Visibility Order Threshold
89	15	100

Figura 34 - Periodo 4: parametri

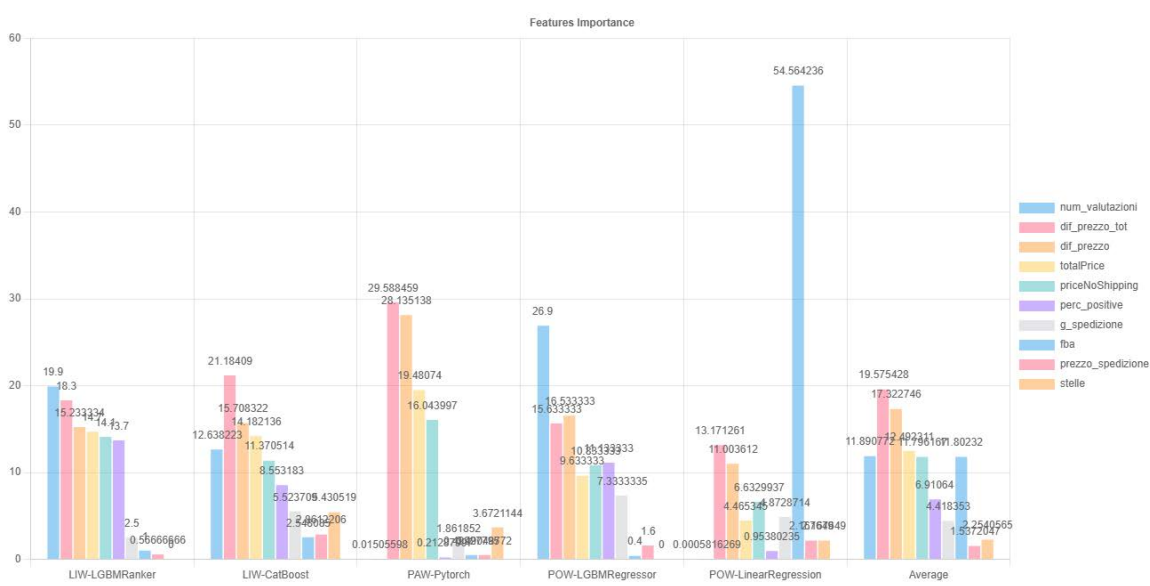


Figura 35 - Periodo 4: grafico delle caratteristiche

Caratteristica	LIW-LGBMRanker	LIW-CatBoost	PAW-Pytorch	POW-LGBMRRegressor	POW-LinearRegression	Average
num_valutazioni	19.9	12.638223	0.01505598	26.9	0.0005816269	11.890772
dif_prezzo_tot	18.3	21.18409	29.588459	15.633333	13.171261	19.575428
dif_prezzo	15.233334	15.708322	28.135138	16.533333	11.003612	17.322746
totalPrice	14.7	14.182136	19.48074	9.633333	4.465345	12.492311
priceNoShipping	14.1	11.370514	16.043997	10.833333	6.6329937	11.796167
perc_positive	13.7	8.553183	0.21287997	11.133333	0.95380235	6.91064
g_spedizione	2.5	5.523709	1.861852	7.333335	4.8728714	4.418353
fba	1	2.548085	0.49927795	0.4	54.564236	11.80232
prezzo_spedizione	0.56666666	2.8612206	0.49048772	1.6	2.167649	1.5372047
stelle	0	5.430519	3.6721144	0	2.167649	2.2540565

Figura 36 - Periodo 4: tabella delle caratteristiche

La figura 35 mostra sotto forma di grafico le informazioni che sono riportate in forma tabellare nella figura 36.

Ndcg	Valore
LIW-CatBoost	0.99724394
LIW-LGBMRanker	0.99900836
POW-LinearRegression	0.9711246
PAW-Pytorch	0.99438184
POW-LGBMRegressor	0.998821

Figura 37 -Periodo 4: NDCG

In figura vengono presentati i risultati NDCG. Tutti gli algoritmi hanno ottenuto risultati superiori al 97%.

4.5.5 Analisi complessiva su tutti i periodi

Tramite le funzionalità offerte dalla dashboard sono stati analizzati tutti i periodi senza impostare nessun filtro nella sezione periodo.

Esplorazione dati dal al



Numero di estrazione: 32270

Prodotto	Numero minimo di venditori	Numero massimo di venditori
Xiaomi Mi Smart Band 6	71	183
Capsule caffè	32	50
Lampadine Philips	15	36
Agenda moleskine	7	24
INTEGRATORE ALIMENTARE	10	11
INSETTICIDA 2	10	11
LAMPADINE	10	11
DISPOSITIVO ANTIABBANDONO	10	11
CUBO DI RUBIK	10	11
GUANTI	10	11
CUFFIE GAMING	10	11
SCHEDA MICRO SD	10	11
TELECAMERA	0	11
FORNO PIZZA	6	11
CAPSULE CAFFE'	3	10
PORTA CELLULARE AUTO	0	6
INSETTICIDA	0	5

Figura 38 - Periodo completo: Esplorazione dati

Come mostrato in figura 38, sono presenti 32.270 registrazioni e 17 prodotti diversi, telecamera insieme a insetticida e porta cellulare sono i prodotti che hanno il minor numero di venditori, prendiamo in considerazione i prodotti che hanno tra i 10 e gli 11 venditori. I prodotti agenda moleskine, porta cellulare auto, insetticida, capsule caffè, telecamera e forno pizza vengono esclusi dall'analisi, utilizzando la funzionalità specifica come mostrato in figura 39, perché hanno una quantità ridotta di venditori.

Prodotti

☒ Xiaomi Mi Smart Band 6
 ☒ Capsule caffè
 ☒ Lampadine Philips
 ☐ Agenda moleskine
 ☒ GUANTI
 ☒ INSETTICIDA 2
 ☒ CUBO DI RUBIK

☐ TELECAMERA
 ☒ LAMPADINE
 ☐ FORNO PIZZA
 ☒ INTEGRATORE ALIMENTARE
 ☒ DISPOSITIVO ANTIABBANDONO
 ☒ SCHEDA MICRO SD

☒ CUFFIE GAMING
 ☐ CAPSULE CAFFÈ
 ☐ PORTA CELLULARE AUTO
 ☐ INSETTICIDA

Figura 39 - Periodo completo: prodotti

È stata eseguita l'analisi su tutte le caratteristiche e con tutti e cinque gli algoritmi. Parametrizzando il sistema come da figura 40

Seed	PyTorch num Epochs	Visibility Order Threshold
11	15	100

Figura 40 - Periodo completo: parametri

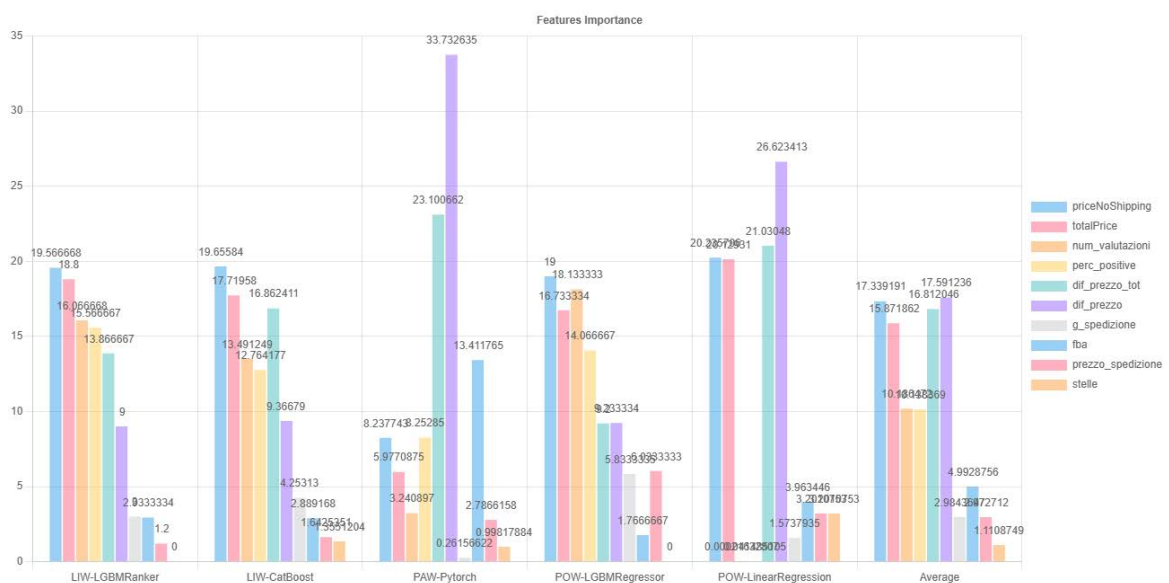


Figura 41 - Periodo completo: grafico delle caratteristiche

Caratteristica	LIW-LGBMRanker	LIW-CatBoost	PAW-Pytorch	POW-LGBMRegressor	POW-LinearRegression	Average
priceNoShipping	19.566668	19.65584	8.237743	19	20.235706	17.339191
totalPrice	18.8	17.71958	5.9770875	16.733334	20.12931	15.871862
num_valutazioni	16.066668	13.491249	3.240897	18.133333	0.00021632807	10.186472
perc_positive	15.566667	12.764177	8.25285	14.066667	0.041485105	10.138369
dif_prezzo_tot	13.866667	16.862411	23.100662	9.2	21.03048	16.812046
dif_prezzo	9	9.36679	33.732635	9.233334	26.623413	17.591236
g_spedizione	3	4.25313	0.26156622	5.8333335	1.5737935	2.9843647
fba	2.9333334	2.889168	13.411765	1.7666667	3.963446	4.9928756
prezzo_spedizione	1.2	1.6425351	2.7866158	6.0333333	3.2010753	2.972712
stelle	0	1.3551204	0.99817884	0	3.2010753	1.1108749

Figura 42 - Periodo completo: tabella delle caratteristiche

La figura 41 mostra sotto forma di grafico le informazioni che sono riportate in forma tabellare nella figura 42.

Ndcg	Valore
LIW-CatBoost	0.99864054
LIW-LGBMRanker	0.99850655
POW-LinearRegression	0.9980892
PAW-Pytorch	0.9998024
POW-LGBMRegressor	0.99990433

Figura 43 - Periodo completo: NDCG

In figura 43 vengono presentati i risultati NDCG. Tutti gli algoritmi hanno ottenuto risultati superiori al 99%.

4.5.6 Confronto dei risultati tra periodi e variazione nel tempo

Per il confronto dei risultati tra i quattro periodi si è deciso di utilizzare la media (colonna average nelle tabelle di risultati) per confrontare l'andamento.

Caratteristica	Periodo 1	Periodo 2	Periodo 3	Periodo 4
num_valutazioni	16,650745	14,126028	10,490812	11,890772
priceNoShipping	15,880560	17,213688	14,406209	11,796167
dif_prezzo_tot	10,843585	8,695223	14,729563	19,575428
totalPrice	16,814692	15,603849	15,392888	12,492311
dif_prezzo	11,543904	9,221727	13,231280	17,322746
perc_positive	5,652905	13,978430	7,292528	6,91064
g_spedizione	3,404291	3,478513	4,148838	4,418353
prezzo_spedizione	4,818927	5,232656	2,837676	1,5372047

fba	12,324766	8,465916	14,645332	11,80232
stelle	2,065622	3,983971	2,824874	2,2540565

Il grafico mostra l'andamento nel tempo dell'importanza delle caratteristiche

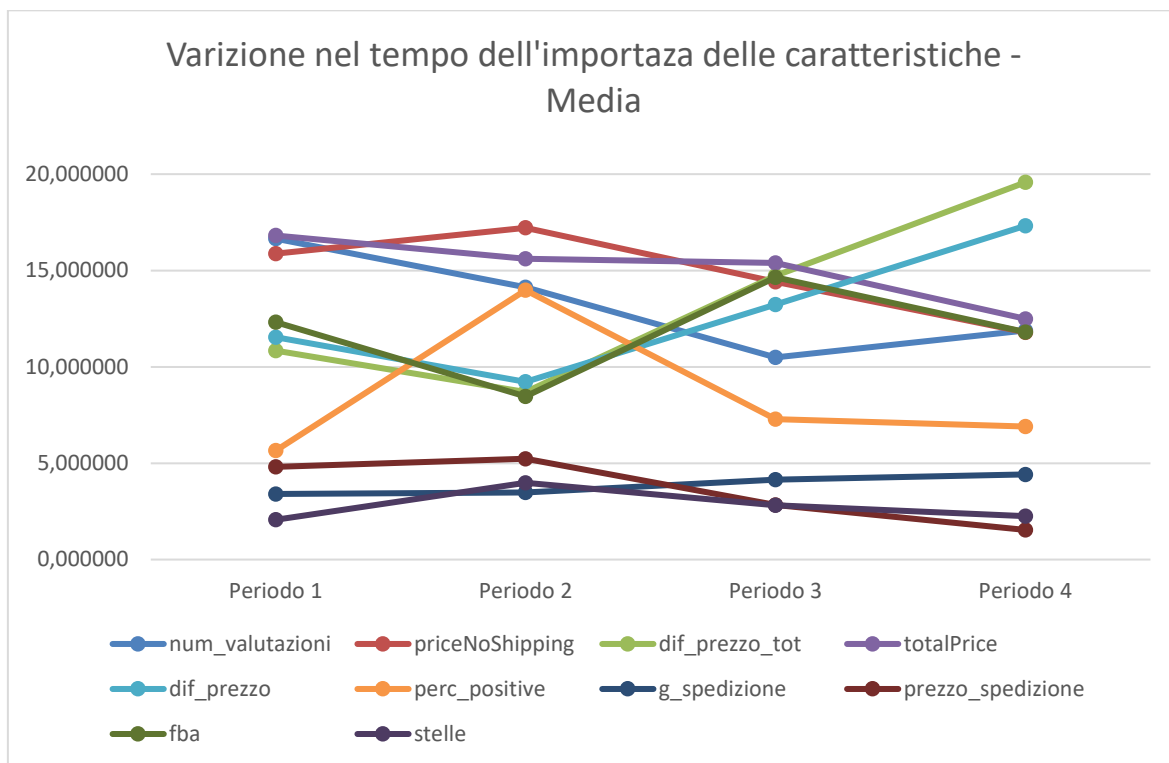


Figura 44 - Variazione del peso delle caratteristiche nei periodi (Media)

Si può osservare come la caratteristica dif_prezzo_tot e dif_prezzo sono quelle che hanno avuto la massima variazione e sono anche le più importanti, mentre priceNoShipping e totalPrice sono quelle che hanno avuto una flessione. Il peso complessivo delle altre caratteristiche rimane abbastanza invariato nei quattro periodi.

Conclusioni e sviluppi futuri

La presente sperimentazione si basa su attività precedenti svolte da altri studenti, delle quali evolve l'architettura e riutilizza parzialmente i dati raccolti.

L'approccio differente è stato di concentrarsi sul ranking dei venditori e non solo sul vincitore della buy box, questo ha permesso di osservare come le singole caratteristiche influenzano il posizionamento in questa classifica, tali posizionamenti possono influire significativamente sul volume di vendite realizzato dai venditori.

È stato dimostrato che è possibile definire un'architettura modulare basata sull'integrazione di tecnologie eterogenee come angular, java e python distribuite tramite container docker e con l'utilizzo di tecnologie standard del web di comunicazione riuscendo ad implementare un unico prodotto. La stessa architettura è utilizzabile in contesti diversi e con carichi di dati diversi, in questa sperimentazione è stata utilizzata la modalità single node, ma i moduli sono stati progettati e realizzati per essere distribuiti su nodi differenti e non omogenei.

L'implementazione e l'utilizzo di un'interfaccia interattiva, ha reso possibile la selezione dei dati e la configurazione dei parametri tramite un'interfaccia web basata su cloud.. I risultati delle elaborazioni sono stati resi disponibili attraverso la medesima interfaccia permettendo di avere tutte le informazioni utili in un unico strumento.

Il sistema, ottenendo risultati con un'elevata accuratezza, ha dimostrato la capacità di modellare efficacemente il ranking dei venditori Amazon. Il modello di analisi realizzato potrebbe essere applicato ad altri e-commerce modificando il modulo di scraping e i relativi metadati estratti.

Dalle informazioni elaborate emerge che le caratteristiche:

- Diff_Prezzo_totale
- Diff_Prezzo
- TotalPrice
- PriceNoShipping
- FBA

Influenzano il ranking in modo maggiore rispetto ad altre, l'importanza di queste caratteristiche è leggermente cambiata nel tempo, questa variazione potrebbe essere dovuta a cambi nel modello utilizzato da Amazon o all'utilizzo di differenti prodotti nell'analisi.

I risultati ottenuti confermano la possibilità di riuscire a reverse-engineerizzare almeno in parte la logica della scelta di presentazione dei venditori da parte di Amazon. È importante sottolineare che l'analisi è stata effettuata solo sulle informazioni ricavabili dalle pagine e non da tutte le informazioni in possesso di Amazon, che potrebbe di conseguenza utilizzare dei criteri differenti, rispetto a quelli evidenziati da questa sperimentazione.

Gli sviluppi futuri potrebbero includere:

- Analisi su nuove caratteristiche del dataset
- Estensione a nuovi prodotti
- Integrazione di dati di tipo immagine o testuale
- Deploy distribuito multi-nodo per alta scalabilità
- Aggiunta di ulteriori modelli

L'attuale architettura a container con deploy dei container tramite docker compose potrebbe essere portata su un orchestratore come kubernetes. Utilizzando kubernetes si avrebbe l'auto-scaling orizzontale dei pod in base al carico, distribuzione del carico su nodi disponibili facendo girare i container sui nodi con il maggior numero di risorse disponibili.

La dashboard potrebbe essere estesa nei parametri di configurazione da passare ai servizi di calcolo, ad esempio aggiungendo la possibilità di definire la percentuale di split tra i dati di train e quelli di test. Si potrebbe valutare anche di aggiungere la possibilità applicare dei filtri oltre che sul range temporale e sui prodotti anche sulle altre caratteristiche presenti nel dataset. L'evoluzione potrebbe includere anche la possibilità di selezionare in ogni elaborazione il set di caratteristiche da esaminare. Queste evoluzioni permetterebbero di avere una dashboard più flessibile e di effettuare analisi differenziate senza dover intervenire sul codice.

5. Bibliografia

Riferimenti bibliografici

[1] Capannini G., Lucchese C., Nardini F. M., Orlando S., Perego R., Tonellotto N.

Quality versus efficiency in document scoring with learning-to-rank models

<https://arpi.unipi.it/handle/11568/1014112>

[2] Chris J.C. Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, Greg Hullender

Learning to Rank using Gradient Descent

<https://www.microsoft.com/en-us/research/publication/learning-to-rank-using-gradient-descent/>

[3] Malay Haldar, Daochen Zha, Huiji Gao, Liwei He, Sanjeev Katariya

Beyond Pairwise Learning-To-Rank At Airbnb

<https://arxiv.org/abs/2505.09795>

Siti web consultati

[1] Stockanalysis, consultato in data 27/04/2025

<https://stockanalysis.com/stocks/amzn/revenue/>

[2] Mobiloud, consultato in data 27/04/2025

<https://www.mobiloud.com/blog/amazon-statistics>

[3] Medium, consultato il 15/04/2025

<https://tamaracucumides.medium.com/learning-to-rank-with-lightgbm-code-example-in-python-843bd7b44574>

[4] Angular.dev, consultato il 26/03/2025

<https://angular.dev/>

[5] Docker, consultato il 28/03/2025

<https://www.docker.com/>

[6] CatBoost, consultato il 07/04/2025

<https://catboost.ai/>

[7] PyTorch, consultato il 10/04/2025

<https://pytorch.org/>

[8] Amazon Titan consultato il 19/06/2025

<https://aws.amazon.com/it/bedrock/amazon-models/titan/>

[8] Amazon Investor Relations consultato il 19/06/2025

<https://ir.aboutamazon.com/news-release/news-release-details/2025/Amazon-com-Announces-First-Quarter-Results/default.aspx>

Indice delle figure

Figura 1 - Amazon: dettaglio prodotto	7
Figura 2 - Amazon: elenco altri venditori	8
Figura 3 - Architettura applicativa.....	17
Figura 4 - Architettura container	18
Figura 5 - Dashboard	32
Figura 6 - dashboard: selezione periodo.....	33
Figura 7 - Dashboard: esplorazione dati.....	33
Figura 8 - Dashboard: grafico delle caratteristiche	34
Figura 9 - Dashboard: tabella delle caratteristiche	34
Figura 10 - Dashboard: grafico delle caratteristiche (Trasposto)	35
Figura 11 - Dashboard: tabella delle caratteristiche (Trasposto).....	35
Figura 12 - Dashboard: ndcg	35
Figura 13 - Rapporto ML- DL.....	37
Figura 14 - Rete neurale	38
Figura 15 - Linear regression	47
Figura 16 - Periodo 1: esplorazione dei dati.....	49
Figura 17 - Periodo 1: parametri	50
Figura 18 - Periodo 1: grafico delle caratteristiche	50

Figura 19 - Periodo 1: tabella delle caratteristiche	50
Figura 20 - Periodo 1: NDCG	51
Figura 21 - Periodo 2: esplorazione dati.....	51
Figura 22 - Periodo 2: parametri	52
Figura 23 - Periodo 2: grafico delle caratteristiche	52
Figura 24 - Periodo 2: tabella delle caratteristiche	52
Figura 25 - Periodo 2: NDCG	53
Figura 26 - Periodo 3: esplorazione dati.....	54
Figura 27 - Periodo 3: prodotti	54
Figura 28 - Periodo 3: parametri	55
Figura 29 - Periodo 3: grafico delle caratteristiche	55
Figura 30 - Periodo 3: tabella delle caratteristiche	55
Figura 31 - Periodo 3: NDCG	56
Figura 32 - Periodo 4: Esplorazione dati	57
Figura 33 - Periodo 4: prodotti	57
Figura 34 - Periodo 4: parametri	58
Figura 35 - Periodo 4: grafico delle caratteristiche	58
Figura 36 - Periodo 4: tabella delle caratteristiche	58
Figura 37 -Periodo 4: NDCG	59
Figura 38 - Periodo completo: Esplorazione dati	60
Figura 39 - Periodo completo: prodotti	61
Figura 40 - Periodo completo: parametri.....	61
Figura 41 - Periodo completo: grafico delle caratteristiche	61
Figura 42 - Periodo completo: tabella delle caratteristiche	62
Figura 43 - Periodo completo: NDCG.....	62
Figura 44 - Variazione del peso delle caratteristiche nei periodi (Media)	63